An Agent-Based Study Assistant with OCR, Retrieval, and Speech

Candidate

Antonis Geralis

Final Year Project submitted in partial fulfilment of the requirements for the Degree of

Bachelor of Science in Computer Science

Department of Computer Science

School of Sciences and Engineering

University of Nicosia

May 2026

Acceptance Page

An Agent-Based Study Assistant with OCR, Retrieval, and Speech

By

Antonis Geralis

This Final Year Project has been accepted in partial fulfilment of the requirements for the Degree of

Bachelor of Science in Computer Science

Role	Name	Signature	Date
Project Advisor	Ioannis Katakis	_____	_____
Examiner	Athena Stassopoulou	_____	_____
Examiner	Vaso Stylianou	_____	_____

Department of Computer Science

School of Sciences and Engineering

University of Nicosia

Abstract

This project presents `study-assistant`, an agent-based study support system designed to help students transform educational material into more usable learning artefacts. The system addresses the practical problem that study resources often exist in fragmented forms, such as scanned documents, lecture notes, long readings, and technical text that must be extracted, organised, queried, and reformulated before effective revision can begin.

The proposed system uses a modular architecture in which an agent coordinates specialised tools for optical character recognition, retrieval-augmented generation, and text-to-speech synthesis. The OCR component converts PDF-based study material into structured text. The retrieval component stores and searches semantically organised study content so that relevant passages can be recovered for question answering, explanation, and revision tasks. The speech component converts prepared text into audio, supporting alternative modes of study. Each tool is designed to operate both as part of the agent workflow and as an independent command-line utility.

The work contributes a cohesive design for an agent-driven educational assistant, a set of interoperable processing tools, and an evaluation of their reliability, ordering behaviour, throughput, and model suitability. The resulting system demonstrates how OCR, semantic retrieval, and speech synthesis can be combined into a practical study workflow while preserving modularity, reproducibility, and clear processing contracts.

Keywords: study assistant, OCR, retrieval-augmented generation, text-to-speech

Acknowledgements

I acknowledge the academic staff of the Department of Computer Science at the University of Nicosia for the guidance and assessment framework supporting this final-year project. I also acknowledge the open-source projects and model providers referenced in this report, whose documentation and interfaces made the implementation and evaluation of the study-assistant system possible.

Table of Contents

List of Tables

List of Figures

Appendices

1 Introduction

1.1 Background and Motivation

Effective university study is an active process of selecting, organising, retrieving, explaining, and testing knowledge. Students do not normally revise from a single clean source. They work across lecture slides, scanned pages, textbook extracts, laboratory sheets, course notes, handwritten or copied summaries, past questions, and fragments collected during teaching. The difficulty is not only that this material may be long or distributed across files. The deeper problem is that study requires repeated transformation of the same material into different learning forms: readable notes, searchable passages, explanations, flashcards, practice questions, essay plans, and revision audio.

This matters because successful study depends on more than passive rereading. Research in cognitive and educational psychology identifies practice testing and distributed practice as high-utility learning techniques, while techniques such as rereading and highlighting are generally weaker when used alone. (Dunlosky et al., 2013) The testing effect literature also shows that retrieval practice can improve long-term retention rather than merely measure existing knowledge. (Roediger & Karpicke, 2006) A study assistant for college students should therefore support workflows that help learners move from source material to active recall, explanation, and self-assessment. It should not be limited to producing summaries.

Students also face a practical access problem. Important course material is often locked in inconvenient formats. A scanned handout cannot be searched reliably. A long set of lecture notes cannot be inserted into a single prompt without losing control over context. A dense technical passage may be difficult to revise while travelling or away from a screen. A student preparing for an exam may need to ask, “What does my material say about this topic?”, “Can

this be turned into flashcards?", "Can I listen to this section?", or "Can I practise with questions based only on these notes?" These are not isolated tasks. They are connected stages in a study workflow.

Modern AI systems are relevant to this problem because language models, document recognition, semantic retrieval, and speech synthesis can each support part of the workflow. However, a general conversational interface alone is not enough. If the assistant is not grounded in the student's own material, it may generate plausible but irrelevant explanations. If scanned documents are not converted into text, the system cannot reason over them. If retrieval is not available, long collections of notes become difficult to query precisely. If output is only visual text, some forms of revision and accessibility are not supported. The project is motivated by the need to coordinate these capabilities around the actual practices of student learning.

1.2 Problem Statement

The problem addressed in this report is the design of an AI-assisted study system that can support college students working with their own educational material. The system must address three connected barriers.

The first barrier is input readiness. Many useful resources are not immediately usable by a language-based assistant. Optical character recognition is necessary when content is present as page images rather than machine-readable text. OCR is therefore not an end in itself; it is the entry point that allows scanned or PDF-based material to participate in later study workflows.

The second barrier is source-grounded access. Students often need help with material that is too large or too fragmented for direct use in a single interaction. Retrieval-augmented generation is relevant here because it combines language generation with an external store of retrieved passages. Lewis et al. describe RAG as a way to combine parametric model

knowledge with non-parametric memory for knowledge-intensive tasks, improving the ability to use specific retrieved evidence during generation. (Lewis et al., 2020) In a study context, this principle supports asking questions over prepared notes, locating relevant course passages, and generating outputs that remain connected to the student's own source material.

The third barrier is output form. Study does not have a single target representation. A student may need verbatim transcription when checking extracted content, a lecture-style explanation when learning a difficult topic, an ELI5 explanation when building intuition, flashcards for retrieval practice, a quiz for self-assessment, essay questions for exam preparation, or audio for listening-based revision. Text-to-speech is especially relevant because learning can involve both visual and auditory channels. Multimedia learning research recognises that modality and presentation form can affect cognitive load and learning under appropriate conditions. (Mayer, 2009)

The central design problem is therefore not simply to connect a language model to a document. It is to provide a coordinated assistant that helps a student move through a complete study cycle: prepare the material, retrieve relevant content, transform it into an appropriate learning artefact, and support revision in more than one mode.

This project presents `study-assistant`, an agent-based study assistant designed for source-grounded academic revision. The primary user is a college student who already has course material and wants to turn it into useful study outputs. The system is intended to support the work that happens between receiving course content and sitting an examination: extracting text, cleaning it, storing it for retrieval, querying it, generating explanations, producing active-recall artefacts, and converting selected material into speech.

At a high level, the system coordinates three capabilities. OCR-based input handling makes scanned or PDF-based material available as text. Retrieval-augmented assistance enables the student to search and reuse prepared study material when asking questions or generating

study artefacts. Text-to-speech output enables selected material to become audio for listening-based revision. The agent provides the user-facing study workflow: it maps the student's intention to an appropriate mode such as transcription, study notes, lecture explanation, simplified explanation, flashcards, mind map, quiz, essay practice, retrieval, or audio preparation.

The purpose of the system is not to replace learning, teaching, or assessment. Its purpose is to reduce the friction between raw course material and effective study activity. The assistant supports the student in preparing material for learning, but the student remains responsible for understanding, checking, and applying the content. This distinction is important in education: AI systems can provide useful support, but they must be framed as aids to learning rather than substitutes for academic engagement.

AI-assisted learning systems are relevant because they can offer flexible access to explanations, practice material, and feedback-like interactions. Work on intelligent tutoring systems has shown that computer-based tutoring can have positive effects in higher education settings, although pedagogy and learning design remain important. (Steenbergen-Hu & Cooper, 2014) More recent discussion of large language models in education identifies opportunities for personalised learning support and content generation, while also emphasising risks such as reliability, overreliance, and responsible use. (Kasneci, 2023)

This project is positioned within that balanced view. It uses AI capabilities to support concrete study tasks, but it constrains the assistant around prepared educational material and explicit study modes. This is especially important for college students, because their learning goals are tied to particular lectures, textbooks, assignments, and examination expectations. A generic answer may be fluent but educationally unhelpful if it does not match the course material. Source-grounded retrieval and controlled output modes help align the assistant with the student's actual learning context.

The agent-based structure is also relevant. A study workflow is naturally multi-step: a student may extract a chapter from a scanned PDF, store it, ask for the key concept behind a topic, generate flashcards, and then produce spoken revision notes. Treating these activities as one coordinated system reduces the burden on the student to manually move material between unrelated tools. The agent acts as the organising layer that connects the learning intention to the appropriate capability.

1.3 Aim and Objectives

The aim of this project is to design and evaluate an agent-based study assistant that supports realistic student revision workflows using OCR, retrieval-augmented assistance, and text-to-speech. The system is intended to make course material easier to access, search, transform, and review while keeping generated outputs grounded in prepared source content.

The objectives are:

- to define the study-assistance problem from the perspective of college students preparing and revising course material;
- to design an agent-based workflow that coordinates input preparation, retrieval, explanation, active-recall generation, and audio revision;
- to support multiple study artefacts, including transcription, study notes, lecture-style explanation, simplified explanation, flashcards, mind maps, quizzes, and essay practice;
- to maintain source-grounded behaviour so that generated outputs are based on the student's material rather than unsupported external content;
- to support both integrated agent use and direct use of individual capabilities where a student only needs one stage of the workflow; and
- to evaluate the system's reliability and practical suitability for study preparation.

1.4 Scope

The project focuses on study preparation and revision for students working with digital or digitised educational material. It is concerned with practical learning workflows rather than broad institutional learning management. The intended setting is a student who has course material and needs assistance turning it into searchable, explainable, testable, or listenable forms.

The system does not attempt to model a complete pedagogy, grade student work, replace instructors, or guarantee subject mastery. It also does not claim that generated study artefacts are automatically correct without student review. Its scope is the design of a modular AI-assisted workflow that helps students prepare and use their own material more effectively.

1.5 Contributions

The main contribution of the project is a cohesive design for an agent-based study assistant that connects document extraction, semantic retrieval, study-output generation, and speech-based revision. The contribution is both conceptual and practical: the system frames AI assistance around the actual sequence of tasks a student performs when preparing for learning and assessment.

The report contributes:

- a student-centered problem formulation for AI-assisted study workflows;
- a high-level architecture for coordinating OCR, retrieval-augmented assistance, and text-to-speech;
- a study interaction model covering source preparation, retrieval, explanation, active recall, and audio revision;
- an evaluation of the system's reliability and practical behaviour; and
- a discussion of the role and limits of agent-based assistance in academic study.

1.6 Thesis Structure

The remaining chapters follow a standard software engineering thesis structure. Chapter Section 2 introduces OCR, retrieval-augmented generation, text-to-speech, agentic tool use, and related AI study systems. Chapter Section 3 defines the system requirements and acceptance criteria. Chapter Section 4 presents the architecture used to coordinate agent-level workflows with specialised processing tools. Chapter Section 5 documents the implemented components. Chapter Section 6 evaluates tool contracts, reliability, OCR performance, model suitability, and end-to-end workflows. Chapter Section 8 discusses the findings, contributions, practical implications, and limitations. Chapter Section 9 summarises the results and future work.

2 Background and Related Work

The study-assistant system combines three established areas of computer science and machine learning: optical character recognition (OCR), retrieval-augmented generation (RAG), and text-to-speech (TTS). The system engineering decisions in the design and implementation chapters are grounded in these fields rather than in ad hoc automation.

The relevant background also includes agentic tool use, structured data interchange, concurrent model calls, and comparable AI-assisted study systems.

2.1 Optical Character Recognition

Optical character recognition is the task of converting visual representations of text into machine-readable symbolic text. Classical OCR systems usually combine image preprocessing, layout analysis, text-line or character segmentation, feature extraction, classification, and post-processing. Smith's overview of the Tesseract OCR engine describes a practical open-source OCR architecture built around connected component analysis, adaptive classification, linguistic processing, and page segmentation. (Smith, 2007)

Modern document OCR is broader than isolated character recognition. Real documents contain tables, equations, multi-column layouts, headers, footers, marginalia, images, and scanned artefacts. Surveys of deep-learning OCR and document understanding describe a shift from hand-engineered recognition pipelines toward learned models that combine computer vision and natural language processing for document-level understanding. (Subramani et al., 2020) Scene text and document text recognition surveys similarly identify layout, perspective, noise, and reading-order ambiguity as persistent sources of difficulty. (Chen et al., 2022; Liu et al., 2019)

For PDF documents, OCR must also account for the distinction between the logical PDF object model and the rendered page. The PDF 2.0 specification defines a page-description format containing text objects, vector graphics, images, fonts, transformations, and rendering instructions. (Standardization, 2020) A page can therefore be visually readable while still lacking reliable extractable text. This is the reason the implemented OCR subsystem adopts a render-first design: `pdfocr` renders each selected page to pixels before model inference.

The olmOCR line of work is especially relevant to this project because it treats PDF extraction as a vision-language problem. The olmOCR paper presents PDF processing as the conversion of visually complex pages into clean, linearised text in natural reading order while preserving structures such as sections, tables, lists, and equations. (Poznanski, 2025) The olmOCR 2 work further frames OCR evaluation through targeted unit-test rewards for document OCR, reflecting the need to test not only raw character accuracy but also structure-sensitive behaviour. (Poznanski et al., 2025)

In this project, OCR is not a final goal by itself. It is a grounding stage for study workflows. If extracted text omits definitions, equations, table content, or reading-order structure, downstream study notes and quizzes become unreliable. This makes recall, structure preservation, and stable page ordering central evaluation criteria.

2.2 OCR Evaluation Concepts

OCR quality is commonly evaluated through edit-distance based measures such as character error rate (CER) and word error rate (WER). These metrics compare a recognised string with a reference transcription. In formula form:

$$\mathrm{CER} = \frac{S_c + D_c + I_c}{N_c} \tag{1}$$

where S_c, D_c, and I_c are character substitutions, deletions, and insertions, and N_c is the number of reference characters. WER uses the same structure over word tokens. These metrics

are valuable because they quantify literal transcription accuracy, but they do not fully capture reading order, table structure, equation fidelity, or semantic completeness. For this reason, the OCR benchmark also reports metrics such as reading-order F1, math-symbol F1, and recall-oriented aggregates.

The design implication is that the OCR subsystem should produce auditable page records. `pdfocr` emits one JSON object per page, including attempts and structured errors, so that evaluation can relate failures to individual pages rather than treating a document as a single opaque output.

2.3 Retrieval-Augmented Generation

Retrieval-augmented generation is a class of methods that combines a parametric language model with an external non-parametric memory. Lewis et al. introduced RAG for knowledge-intensive NLP tasks, combining a pre-trained sequence-to-sequence generator with a dense vector index of retrieved passages. (Lewis et al., 2020) The motivation is that a language model's parameters alone are an imperfect store of factual knowledge: retrieval can provide more specific evidence, support provenance, and allow knowledge updates without retraining the generator.

In the canonical RAG formulation, a query x is used to retrieve candidate passages z from a corpus. A generator then produces output y conditioned on both the query and retrieved passages. A simplified RAG-sequence objective can be written as:

$$p(y \mid x) \approx \sum_{z \in \text{ top\_k}(x)} R(z \mid x) G(y \mid x, z) \tag{2}$$

where R is the retriever distribution and G is the generator distribution. The implemented system does not train a joint RAG model; instead it implements the engineering substrate for retrieval-grounded study workflows: documents are chunked, embedded, stored, retrieved, and then used by the agent as source-grounding material.

Dense passage retrieval provides the retrieval model underlying many RAG systems. Karpukhin et al. show that dual-encoder dense representations can retrieve candidate passages for open-domain question answering and can outperform strong sparse baselines on several datasets. (Karpukhin et al., 2020) This supports the design of `chunkvec`: source chunks and queries are both embedded as numeric vectors, and nearest-neighbour search returns semantically related passages.

2.4 Embeddings and Vector Search

An embedding model maps text into a vector space:

$$f : \texttt{text} \rightarrow \mathbb{R}^d \tag{3}$$

The retrieval task is to find stored chunks whose vectors are close to a query vector. A common similarity measure is cosine similarity:

$$\cos(q, v) = \frac{q \cdot v}{\|q\|\|v\|} \tag{4}$$

The implementation uses an OpenAI-compatible embeddings endpoint to produce float vectors and stores them in SQLite. `sqlite-vector` provides vector search functions over vectors stored as BLOBs in ordinary SQLite tables. Its API requires fixed vector dimensions for an initialized vector column and a quantization step before quantized scanning. (sqliteai, 2026)

The study-assistant use case places special importance on chunking. A chunk should be large enough to preserve context, but small enough to retrieve a focused passage. This aligns with the information retrieval principle that retrieval units determine what evidence can be returned. In this project, the `rag-tool` instruction layer enforces semantic chunk boundaries and stable metadata so that retrieved chunks remain useful as study evidence.

2.5 Text-to-Speech

Text-to-speech converts written text into speech audio. Traditional TTS systems are often described in terms of front-end linguistic processing, acoustic modelling, and waveform generation. Modern neural TTS systems learn much of this mapping using sequence-to-sequence and neural vocoder architectures.

WaveNet introduced an autoregressive neural model for raw audio generation and showed its applicability to TTS. (Oord, 2016) Tacotron 2 combines a sequence-to-sequence model that predicts mel spectrograms from character input with a WaveNet vocoder that synthesises the waveform, achieving speech quality close to professional recordings under the reported mean opinion score evaluation. (Shen et al., 2018) These systems establish the modern framing used by model-hosted TTS APIs: input text is converted by a neural model into audio samples or encoded audio.

The implementation in this project does not train a TTS model. It uses an OpenAI-compatible audio speech endpoint and focuses on the engineering problems around reliable generation:

- rewriting visual text into speech-friendly prose;
- splitting long text into bounded spoken units;
- issuing bounded concurrent speech requests;
- validating returned audio bytes;
- preserving chunk order; and
- writing one final `.opus` artefact only when generation succeeds.

This is a practical application of neural TTS as a service. The technical contribution lies in preparation, orchestration, validation, and artefact construction.

2.6 Agentic Tool Use

The study assistant is an agentic system in the practical software-engineering sense: it selects among tools based on user intent and intermediate artefact state. The agent does not subsume OCR, retrieval, and speech synthesis into one prompt. Instead, it composes specialised subsystems with explicit contracts. This design is consistent with the broader principle of modular AI systems: use specialised mechanisms for perception, retrieval, generation, and output transformation, and keep their boundaries auditable.

The agent layer is therefore responsible for:

- choosing a study mode;
- deciding whether extraction, retrieval, or speech generation is required;
- preserving source-grounding constraints;
- preparing content for the target workflow; and
- presenting final study artefacts in the appropriate format.

The core tools are responsible for deterministic processing. This separation makes the system easier to test than a monolithic agent because many correctness properties are reduced to command-line contracts, schemas, database invariants, and pipeline state machines.

2.7 Related AI Study Systems

The project is situated within a current class of AI-assisted study and research systems that transform user-provided sources into study artefacts. The comparison here is functional and architectural rather than evaluative; it identifies similarities and distinctions that clarify the position of the proposed system.

Thea.study is an AI study platform that allows students to upload materials and generate study aids such as practice questions, flashcards, study guides, summaries, and adaptive study activities. Its public descriptions emphasise file upload, active recall, spaced

repetition, practice variation, games, and multilingual access. (Thea Study, 2026) Functionally, it overlaps with this project in its student-facing emphasis on transforming course material into revision artefacts.

Google NotebookLM is an AI-powered research and note assistant that allows users to upload sources, chat with notebooks, receive source-grounded responses, and transform sources into formats such as study guides, briefings, audio overviews, and mind maps. Google's documentation emphasises grounding in uploaded sources and inline citations, while its Audio Overview feature generates source-based audio discussions in multiple formats. (Google, 2026a; 2026b)

The present project shares the general objective of helping students or researchers work with their own material. Its distinction is architectural. The Study and NotebookLM are hosted products with integrated user-facing environments; this project is an open, modular academic implementation with explicit standalone OCR, retrieval, and speech components coordinated by an agent-level workflow layer.

This makes the project better suited to the aims of the thesis, although not necessarily better as a commercial product. The implementation exposes the boundaries between orchestration, processing tools, and reusable infrastructure. OCR, retrieval, and speech generation can be tested independently, invoked without the agent, and inspected through command-line inputs, outputs, schemas, logs, and benchmark artifacts. This transparency supports reproducibility and technical evaluation in a way that hosted products do not normally expose to users.

The comparison therefore does not claim superiority in product maturity, user experience, or model capability. It positions the project as a more inspectable and extensible engineering contribution for academic study workflows.

2.8 Streaming, Concurrency, and Structured Data

The system relies on streaming and structured data for composability. JSON Lines is a line-delimited format where each line is a valid JSON value and can be processed independently. (JSON Lines, 2026) This matches the OCR use case: each page produces one record, and downstream processes can consume records incrementally.

The remote model calls are network-bound and therefore benefit from concurrency. The implementation uses libcurl's multi interface through `relay`; the libcurl documentation describes the multi interface as a way for applications to manage multiple simultaneous transfers and drive network progress explicitly. (curl, 2026) This supports the bounded-concurrency design used by `pdfocr`, `chunkvec`, and `chunktts`.

Structured JSON handling is also central. The OpenAI-compatible APIs require request and response bodies with predictable schemas, and the tools need stable result objects. The custom `jsonx` library provides typed JSON mapping and streaming output, which reduces the risk of schema drift compared with ad hoc string construction.

3 Requirements and Specification

3.1 Requirements Elicitation and Organisation

The requirements were drafted by the author from the project study, exploratory workflow experiments, and analysis of common student revision tasks. The specification states the responsibilities of each subsystem and the guarantees that connect them.

The specification is organised around four levels, moving from user-visible behaviour to implementation constraints:

- user-facing study workflows;
- agent-level orchestration rules;
- tool-level command and artefact contracts; and
- core implementation invariants.

3.2 System Actors and Use Cases

The primary actor is a student or researcher preparing study material. The user may provide a PDF, text file, markdown file, stored document identity, or raw text. The agent selects a workflow and uses the relevant tool when the source material requires processing.

The main use cases are:

- transcribe source material into structured markdown while preserving educational content;
- generate professor-style lecture explanations from prepared material;
- produce plain-English explanations without losing technical accuracy;
- create flashcards, Mermaid mind maps, quizzes, and essay prompts;
- extract raw text from PDFs before downstream processing;
- ingest cleaned text into a vector store for retrieval;

- search stored study material using semantic queries and metadata filters; and
- convert prepared study text into natural `.opus` audio.

3.3 Functional Requirements

The agent-level functional requirements are:

- **FR-A1:** The system shall expose `study-assistant` as the coordinating agent for study workflows.
- **FR-A2:** The agent shall map user intent to a single study mode when producing a direct study artefact.
- **FR-A3:** The agent shall operate only on available source material or material explicitly extracted by the tools.
- **FR-A4:** The agent shall preserve source meaning and avoid inventing content during cleaning, chunking, retrieval preparation, and TTS preparation.
- **FR-A5:** The agent shall invoke specialised tools for OCR, RAG, and TTS rather than embedding these concerns into one monolithic prompt.

The OCR functional requirements are:

- **FR-O1:** `ocr-tool` shall own PDF-to-text extraction workflows at the instruction layer.
- **FR-O2:** `pdfocr` shall accept a local PDF and exactly one page-selection mode: selected pages or all pages.
- **FR-O3:** `pdfocr` shall render pages, submit OCR requests to an OpenAI-compatible multimodal model endpoint, and emit one JSON object per selected page.
- **FR-O4:** Successful page results shall include page number, status, attempt count, and extracted text.
- **FR-O5:** Failed page results shall include page number, status, attempt count, error kind, error message, and HTTP status where applicable.

The RAG functional requirements are:

- **FR-R1:** `rag-tool` shall own storage and retrieval workflows at the instruction layer.
- **FR-R2:** `cvstore` shall ingest one marked-up text file whose chunks begin with `<chunk ...>` markers.
- **FR-R3:** `cvstore` shall apply command-line document identity and content kind metadata to the ingest run.
- **FR-R4:** `cvquery` shall embed one query string and retrieve nearest-neighbour matches from the local vector database.
- **FR-R5:** Search shall support metadata filters for document, kind, page, and label where valid.

The TTS functional requirements are:

- **FR-T1:** `tts-tool` shall own text-to-speech workflows at the instruction layer.
- **FR-T2:** The tool shall rewrite visual or technical text into a natural spoken form without changing the underlying meaning.
- **FR-T3:** `chunktts` shall accept one marked-up text file and one output path.
- **FR-T4:** `chunktts` shall split input on `<bk>` markers, synthesise each chunk, validate audio, and write a single final `.opus` file.
- **FR-T5:** The final audio order shall match the normalised chunk order.

3.4 Non-Functional Requirements

The system non-functional requirements are:

- **NFR-1 Modularity:** Agent definitions, tool definitions, and core implementations shall remain separately understandable and usable.
- **NFR-2 Composability:** Command-line tools shall use stable inputs and outputs suitable for shell workflows.
- **NFR-3 Determinism:** Page and chunk ordering shall be deterministic regardless of network completion order.

- **NFR-4 Bounded concurrency:** Remote model calls shall be limited by a configured in-flight bound.
- **NFR-5 Retry robustness:** Transient network and selected API failures shall be retried according to explicit policy.
- **NFR-6 Observability:** Logs and diagnostics shall not pollute machine-readable outputs.
- **NFR-7 Configuration clarity:** API URLs, models, keys, and concurrency settings shall be resolved through documented defaults, optional `config.json`, and environment overrides where supported.
- **NFR-8 Standalone utility:** OCR, RAG, and TTS tools shall remain useful without the agent.

3.5 Requirements Traceability Matrix

Table 1 maps the main requirements to implementation mechanisms.

3.6 Acceptance Criteria

The project is considered complete when the following criteria are met:

- the report explains the agent and tool suite as a single system;
- all seven repositories are represented in the architecture narrative;
- OCR, RAG, and TTS are documented both as standalone tools and as agent components;
- the Nim implementation foundation is clearly described;
- olmOCR 2 results are reported and interpreted in the evaluation section;
- no component is presented as separate from the unified system design.

Requirement	Agent/tool layer	Core implementation mechanism
FR-A2	`study-assistant` mode selection	prepared source text consumed by the agent
FR-O1–FR-O3	`ocr-tool` workflow	`pdfocr` rendering, WebP encoding, OCR request pipeline
NFR-3	`ocr-tool` expects raw ordered text	`pdfocr` sequence ids and staged JSONL emission
FR-R1–FR-R3	`rag-tool` store mode	`cvstore` chunk parser, embeddings pipeline, SQLite insert
FR-R4–FR-R5	`rag-tool` search mode	`cvquery` query embedding and `sqlite-vector` nearest-neighbour search
FR-T1–FR-T5	`tts-tool` workflow	`chunktts` chunk splitting, speech requests, audio validation
NFR-4, NFR-5	tool execution guidance	`relay` `maxInFlight`, retry queues, request-id codecs
FR-O4–FR-O5, NFR-6	tool configuration and outputs	`jsonx` typed parsing and streaming JSON serialisation
FR-O3, FR-R4, FR-T4	model endpoint configuration	custom `openai` chat, embedding, and speech helpers

Table 1: Requirements traceability

4 System Architecture and Design

4.1 Design Overview

The system is designed as a modular agent-driven study platform rather than as a single-purpose OCR program. The main architectural thesis is that study workflows require both flexible orchestration and deterministic processing. The agent layer provides flexibility: it interprets user intent, chooses a study mode, and composes tool workflows. The core tools provide determinism: they expose stable command-line contracts, bounded concurrency, explicit retry behaviour, and auditable artefacts.

The system is therefore organised into five layers:

1. **User interaction layer:** the student request and final study output.
2. **Agent orchestration layer:** the `study-assistant` workflow and mode selection.
3. **Tool definition layer:** `ocr-tool`, `rag-tool`, and `tts-tool` operational instructions.
4. **Core processing layer:** Nim executables implementing OCR, retrieval, and speech synthesis.
5. **Shared infrastructure layer:** local storage, artefact files, HTTP transport, JSON handling, and model API schema support.

This layered design avoids coupling study-output generation to implementation details such as PDFium handles, SQLite vector storage, or libcurl polling. It also keeps tool execution verifiable independently of the agent.

4.2 Global Architecture

Figure 1 shows the architectural layers, local execution boundary, and external provider boundary.

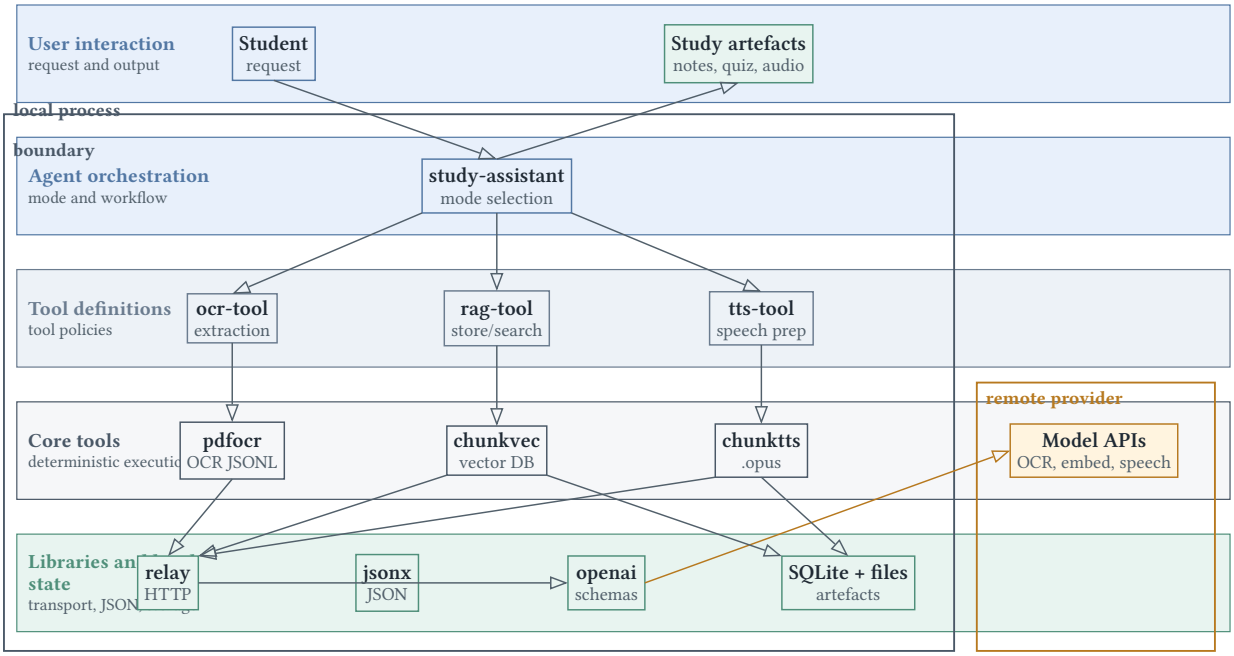

Figure 1: Layered architecture of the study-assistant system with local execution and remote provider boundaries.

The figure expresses the central separation of concerns. The agent and tool definitions are declarative and operational layers. The Nim components implement local deterministic execution. The shared libraries provide common infrastructure, while model-hosted OCR, embedding, and speech endpoints remain outside the local process boundary.

The background concepts from Chapter Section 2 map to concrete component responsibilities: rendering-first OCR is implemented by `pdfocr`, retrieval by `chunkvec`, speech synthesis by `chunktts`, and concurrent model access by the combination of `relay`, `openai`, and typed JSON handling.

4.3 Agent-Level Interaction Model

The `study-assistant` agent defines a mode-selection interface. A user request is mapped to exactly one study-output mode when the output is generated directly from prepared material. The supported modes have distinct contracts:

Repository	Layer	Primary responsibility	Principal artefacts
`study-assistant`	Agent	Select study mode and generate grounded study outputs	mode rules for transcribe, lecture, ELI5, flashcards, mindmap, quiz, essay, notes
`ocr-tool`	Tool definition	Convert PDFs to raw text using `pdfocr`	OCR command rules, cache workflow, extraction cleanup
`rag-tool`	Tool definition	Store/search study material using `chunkvec`	store/search modes, chunking rules, metadata policy
`tts-tool`	Tool definition	Produce speech audio using `chunktts`	speech rewriting rules, chunk boundary policy
`pdfocr`	Core tool	Render PDF pages and perform model-based OCR	ordered JSONL, page errors, retry state machine
`chunkvec`	Core tool	Embed, store, and retrieve study chunks	SQLite vector store, chunk parser, search output
`chunktts`	Core tool	Generate ordered speech audio	speech chunks, audio validation, final `.opus`
`relay`	Library	Execute bounded concurrent HTTP requests	worker thread, libcurl multi, request/result API
`jsonx`	Library	Parse and write typed JSON	parser, object mapper, streaming writer, raw JSON
`openai`	Library	Build and parse OpenAI-compatible API calls	chat, embeddings, audio speech request helpers

Table 2: Component responsibilities

- **transcribe:** preserve educational text verbatim while removing only clear metadata noise;
- **lecture:** present material as formal connected prose;
- **eli5:** explain the same material in simple language without adding unrelated facts;
- **flashcard:** produce a two-column exam-revision table;
- **mindmap:** produce a Mermaid mind map in a strict hierarchy;
- **quiz:** produce mixed practice questions and an answer key;
- **essay:** produce exam-style prompts and sample answers; and
- **study-notes:** produce progressive exam-focused notes.

This mode design is important because it prevents the agent from conflating extraction, explanation, retrieval, and transformation. A transcription mode should not summarise. A quiz mode should not introduce outside knowledge. A TTS preparation step should optimise for speech while preserving meaning.

4.4 Tool Definition Layer

The three tool-definition repositories encode operational policy.

`ocr-tool` owns PDF extraction. It requires `pdfocr` for PDF text extraction, supports page ranges and full-document mode, and defines a cache procedure so repeated PDF/page selections do not require repeated model calls. The cache is keyed by input PDF identity and page selection. This is a tool-level optimisation: it is outside `pdfocr` so the OCR executable remains a stateless command-line tool.

`rag-tool` owns storage and retrieval. It distinguishes store mode from search mode, requires stable document identifiers, and separates global ingest metadata (`doc`, `kind`, `source`) from per-chunk metadata (`page`, `label`). This distinction maps directly onto the `chunkvec` database schema.

`tts-tool` owns speech preparation. It rewrites markdown, links, mathematical notation, URLs, file paths, and punctuation-heavy text into a speakable form. It also decides where `<bk>` markers should be placed. This is intentionally done before `chunktts` because natural speech preparation is a linguistic transformation, while `chunktts` is an artefact-generation pipeline.

4.5 Operational Contracts

`study-assistant` requires prepared source material, maps each request to a single output form, and removes only clear metadata noise while preserving educational content.

`ocr-tool` delegates PDF extraction to `pdfocr`, supports full-document and page-range extraction, and treats extracted text as raw material for downstream processing.

`rag-tool` requires plain text or markdown input, stable document identifiers, explicit chunk boundaries, and retrieval from a reusable vector store.

`tts-tool` requires manual rewriting for natural speech, conservative chunking with `<bk>` markers, and generation through `chunktts`.

Repository	Command surface	System role	Core contract
`pdfocr`	`pdfocr INPUT.pdf --pages:"..."` or `--all-pages`	OCR processing	ordered JSONL page results, bounded concurrency, retry handling
`chunkvec`	`cvstore`, `cvquery`	RAG storage/search	marked chunk ingest, SQLite vector store, local nearest-neighbour search
`chunktts`	`chunktts INPUT.txt OUTPUT.opus`	speech generation	ordered chunk synthesis, audio validation, one final `.opus` artefact

Table 3: Core tool command contracts

4.6 Core Processing Pattern

The three core tools share a common processing pattern:

1. Parse CLI arguments and validate required inputs.
2. Load optional configuration from the executable directory.
3. Resolve API credentials with environment-variable precedence.
4. Normalise input into ordered work items.
5. Assign deterministic request identifiers.
6. Submit bounded batches through `relay`.
7. Classify completions into success, retryable failure, or terminal failure.
8. Retry retryable failures according to a time-ordered retry queue.
9. Finalise output in deterministic order or in a transactional database state.
10. Map the outcome to a stable exit code.

The shared pattern is not accidental. It is the mechanism by which the system keeps remote model calls reliable enough for academic study workflows.

The OCR pipeline uses `N` for selected page count and `K` for maximum active/in-flight work. The staged result array is O(`N`) metadata. WebP payload storage is O(`K`) because only active pages keep cached payload bytes for possible retry. In RAG ingest, the same pattern terminates in transactional database insertion. In TTS, it terminates in all-or-nothing audio publication.

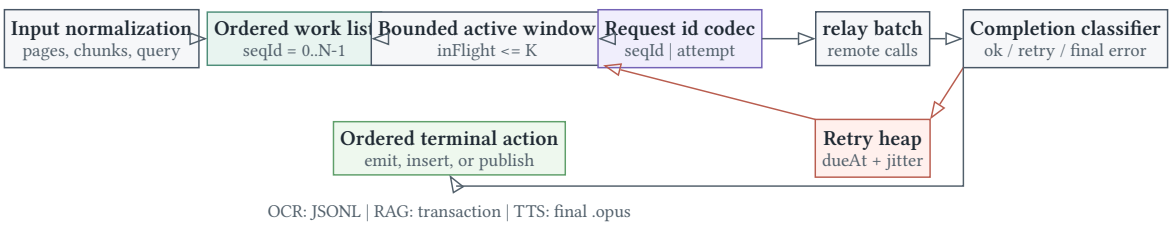

Figure 2: Shared bounded-concurrency execution pattern across OCR, retrieval ingest, and speech generation.

4.7 Retrieval Boundary

The storage path (`cvstore`) and query path (`cvquery`) are deliberately separate. Storage performs remote embedding generation and database mutation. Query performs a single remote query-embedding request, then local vector search. This limits remote dependency during retrieval and keeps the search corpus under local user control.

4.8 Speech Publication Boundary

The TTS pipeline differs from OCR in its publication rule. OCR can emit per-page error records because downstream tools can choose how to handle partial page failures. TTS withholds the final audio file unless all chunks succeed, because a partial lecture audio file would be misleading.

4.9 Relay-Based Interaction Model

The core tools use `relay` to avoid embedding libcurl logic in each processing pipeline. Figure 3 shows the concurrency boundary.

The tool main thread owns ordering, retries, and output state. The relay worker owns transfer execution. This is a narrow concurrency boundary: state that determines correctness remains in the tool, while network multiplexing remains in the transport library.

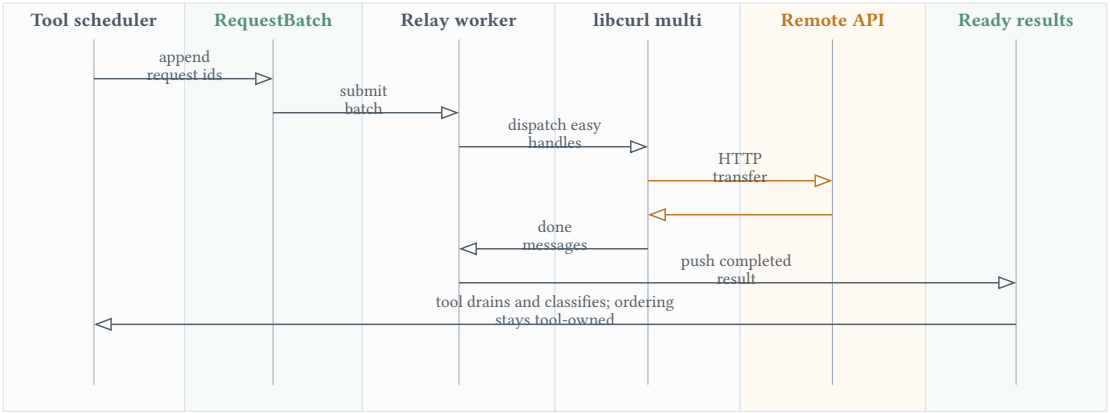

Figure 3: Concurrency boundary between a core tool scheduler and the `relay` transport worker.

4.10 Cross-Cutting Invariants

The architecture depends on several invariants:

- **I1: bounded active work.** Each network-dependent pipeline enforces `inFlightCount <= K`; OCR and TTS also bound active cached payloads or decoded chunks according to the work list.
- **I2: deterministic request identity.** Request ids encode sequence id and attempt, allowing out-of-order completions to be classified without shared mutable lookup tables.
- **I3: ordered finalisation.** OCR emits from `staged[nextEmitSeqId]`; TTS writes decoded chunks in sequence order; RAG stores chunks with stable metadata and searches with deterministic ordering by distance and id.
- **I4: explicit retryability.** Transport errors and selected HTTP statuses are classified before retry; permanent failures are not retried indefinitely.
- **I5: clean process channels.** Machine-readable outputs use stdout where relevant; logs use stderr.
- **I6: configuration normalisation.** Invalid numeric values and empty strings are replaced by defaults so runtime state remains within expected bounds.

These invariants are the bridge between the background concepts in Chapter Section 2 and the implementation details in Chapter Section 5.

4.11 Design Rationale

The design uses standalone tools rather than a single long-running service for three reasons.

First, command-line tools are easy to compose in student workflows. OCR JSONL can be redirected, stored, inspected with `jq`, or ingested into other systems. Generated audio is a normal `.opus` file. The vector database is local.

Second, independent tools make failures easier to isolate. If OCR fails, the failure is represented as page-level JSON or a fatal exit code. If TTS fails, the final audio file is withheld. If retrieval returns irrelevant chunks, the issue can be examined through chunk boundaries and metadata.

Third, the architecture supports agent use without requiring the agent to implement low-level systems. The agent remains responsible for study logic; the tools remain responsible for processing contracts.

5 Implementation

5.1 Implementation Overview

The associated repositories form a coherent implementation with explicit structure, public interfaces, internal modules, algorithms, data flows, and verification surfaces.

The implementation can be grouped into three categories:

- **instruction repositories:** `study-assistant`, `ocr-tool`, `rag-tool`, `tts-tool`;
- **core processing repositories:** `pdfocr`, `chunkvec`, `chunktts`; and
- **shared Nim libraries:** `relay`, `jsonx`, `openai`.

5.2 Agent and Tool Layer

5.2.1 `study-assistant`: Agent Definition Repository

The `study-assistant` repository defines the top-level agent behaviour. Its principal artefact is an instruction file that maps user requests onto study-output modes. The key design decision is that the agent operates on prepared source material. It does not prescribe OCR, vector search, or speech generation internally; instead it relies on supporting tools when the source material requires extraction, retrieval, or audio production.

The agent mode table is:

Mode	Purpose	Output discipline
`transcribe`	preserve source text	structured markdown, no summarisation
`lecture`	explain formally	connected academic prose
`eli5`	simplify	plain language with maintained technical meaning
`flashcard`	revision	two-column markdown table
`mindmap`	concept hierarchy	Mermaid mindmap only
`quiz`	practice	mixed questions with answer key
`essay`	exam preparation	prompts plus sample answers
`study-notes`	revision notes	progressive topic-organised notes

Table 4: Agent study modes and output disciplines

The design algorithm is:

1. Receive a user request and source material.
2. Determine whether the source material is already prepared text.
3. If not, delegate extraction or retrieval to a tool.
4. Select exactly one study-output mode.
5. Clean only obvious metadata noise.
6. Generate the target artefact using only available source content.

The major invariant is source grounding: the agent must not introduce unsupported factual claims into study outputs. This is essential for academic reliability.

Figure 4 shows the capability routing implemented by the instruction layer. The diagram is grounded in the mode-selection rules and the tool handoff conditions in the instruction repositories.

5.2.2 `ocr-tool`: OCR Tool Definition Repository

`ocr-tool` defines when and how PDF extraction is performed. It owns the use of `pdfocr` at the instruction layer. Its workflow is:

1. Determine that the input is a PDF requiring text extraction.
2. Check whether OCR output for the same PDF/page selection is available in the session cache.
3. If the cache misses, run `pdfocr` with `--all-pages` or `--pages:"..."`.

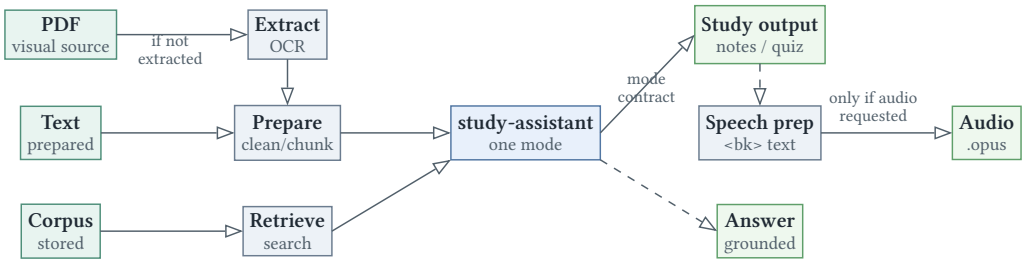

Figure 4: Capability routing across input conditions, tool handoffs, study modes, and optional audio output.

4. Store extracted text in the cache.
5. Pass raw extracted text to the downstream study workflow.

This repository also defines a strict responsibility boundary: OCR extraction does not generate summaries, flashcards, or interpretations. It produces source text. The cleanup rule removes only clear metadata such as headers, footers, page numbers, timestamps, and extraneous identifiers.

The cache design is intentionally outside `pdfocr`. The OCR executable stays stateless and composable, while the tool definition can optimise repeated agent sessions.

5.2.3 `rag-tool`: RAG Tool Definition Repository

`rag-tool` defines two modes: store and search.

In store mode, the agent prepares chunked text. Chunking is semantic rather than layout-driven. A chunk should represent one concept or tightly related concept cluster, contain enough context to stand alone, and avoid mixing unrelated topics. The tool distinguishes:

- global ingest metadata: `doc`, `kind`, `source`;
- per-chunk metadata: `page`, `label`.

This maps directly to `chunkvec`'s storage schema. The design protects retrieval quality by requiring stable `doc` identifiers and controlled labels. It also prevents a common RAG failure mode: chunks that are too small to be meaningful or too large to be specific.

In search mode, the tool builds one semantic query string and applies filters only when the user clearly requests constrained retrieval. This avoids over-filtering, which can hide relevant material.

5.2.4 `tts-tool`: TTS Tool Definition Repository

`tts-tool` defines the transformation from visual text to speech-ready text. The repository treats speech preparation as a separate task from audio synthesis. The agent must rewrite or remove:

- markdown syntax;
- table syntax;
- raw URLs;
- email addresses;
- LaTeX delimiters and common symbols;
- file paths and technical identifiers;
- punctuation-heavy fragments; and
- decorative content with no spoken value.

The output passed to `chunktts` is a text file with `<bk>` markers. These markers represent spoken thought units, not necessarily paragraphs or original document sections. The recommended chunk size is conservative because long TTS chunks are more likely to fail or sound unnatural.

The design separates linguistic preparation from synthesis. `tts-tool` decides what should be spoken; `chunktts` decides how to request, validate, and assemble audio.

5.3 OCR Implementation

5.3.1 `pdfocr`: Module Structure

`pdfocr` is a Nim command-line OCR system. The inspected structure contains:

- `src/app.nim`: application entry point, exit-code mapping, relay shutdown policy;
- `src/pdfocr/runtime_config.nim`: CLI parsing, configuration loading, page-count discovery;
- `src/pdfocr/page_selection.nim`: page-selection grammar and sorted uniqueness;
- `src/pdfocr/pdfium_wrap.nim`: ownership-safe PDFium wrappers;
- `src/pdfocr/pdf_render.nim`: page rendering and WebP handoff;
- `src/pdfocr/webp_wrap.nim`: libwebp encoding wrapper;
- `src/pdfocr/ocr_client.nim`: OCR request construction and response parsing;

- `src/pdfocr/pipeline.nim`: bounded-concurrency OCR state machine;
- `src/pdfocr/request_id_codec.nim`: sequence/attempt packing;
- `src/pdfocr/retry_queue.nim`: heap-based retry scheduler;
- `src/pdfocr/retry_and_errors.nim`: retryability and final error mapping;
- `src/pdfocr/types.nim`: runtime types and JSONL result schema; and
- tests for page selection, request ids, retry logic, retry queue, and JSON output.

The repository is structured around a clean separation between native-resource wrappers, configuration, request construction, scheduling, and output schema.

5.3.2 `pdfocr`: Core Algorithms

The page-selection algorithm parses a comma-separated grammar with singleton pages and inclusive ranges. Each page is inserted into the result sequence using a lower-bound search. This yields sorted unique page numbers regardless of input order or duplicates.

The request-id codec reserves 16 bits for attempt number and uses the remaining positive signed range for sequence id. Formally:

$$\text{requestId} = (\text{seqId} \ll 16) \mid \text{attempt} \tag{5}$$

The codec checks both sequence count and maximum attempts before the pipeline starts. This makes completion matching deterministic and avoids ambiguous request ids.

The pipeline state contains:

- `inFlightCount`;
- `activeCount`;
- `staged`;
- `cachedPayloads`;
- `retryQueue`;
- `nextSubmitSeqId`;
- `nextEmitSeqId`;

- `remaining`;
- `submitBatch`;
- `allSucceeded`; and
- random state for retry jitter.

The main loop performs:

1. submit due retries;
2. submit fresh attempts while capacity remains;
3. start a relay batch;
4. flush ordered staged results;
5. drain ready relay results;
6. flush again;
7. wait for progress if no result was drained.

This loop implements both throughput and ordering. It may receive page 10 before page 2, but it cannot emit page 10 until all earlier selected sequence ids are staged.

Figure 5 presents the page lifecycle implemented by `pipeline.nim`. It captures the actual states represented by payload preparation, in-flight requests, retry scheduling, final staging, and ordered emission.

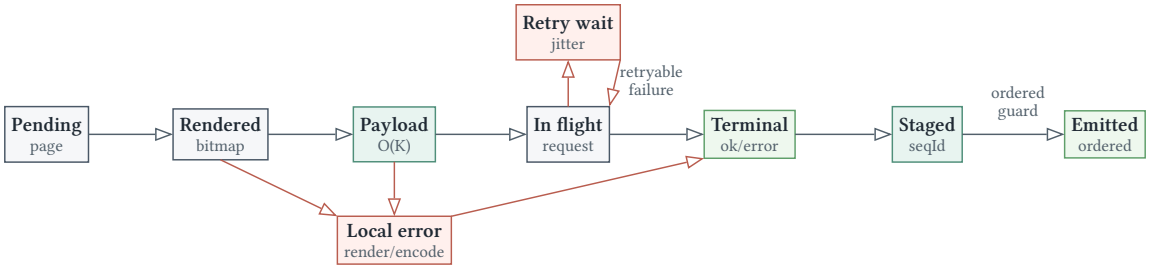

Figure 5: Per-page OCR lifecycle with retry scheduling, bounded cached payloads, staging, and ordered emission.

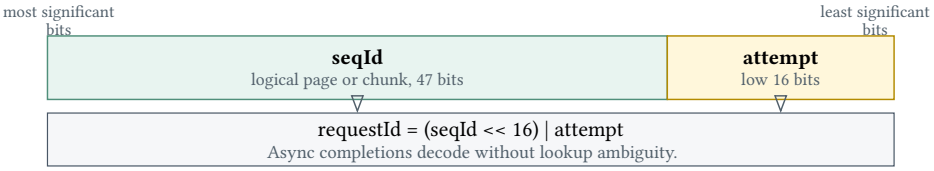

Figure 6: Request-id layout used for deterministic completion matching.

Figure 6 illustrates the request-id packing scheme shared by the core pipelines. This encoding is the mechanism that lets asynchronous completions be mapped back to both the logical item and the attempt number.

5.3.3 `pdfocr`: Error and Output Model

`pdfocr` has item-level and fatal failures. Item-level failures become JSONL objects. Fatal failures become exit code `3` and may leave stdout incomplete.

The page result schema has success and error variants:

The error kinds are `PdfError`, `EncodeError`, `NetworkError`, `Timeout`, `RateLimit`, `HttpError`, and `ParseError`.

5.4 RAG Implementation

5.4.1 `chunkvec`: Module Structure

`chunkvec` implements retrieval storage and query. Its structure contains:

- `src/cvstore.nim`: ingest application entry point;
- `src/cvquery.nim`: search application entry point;

Field	Success	Error	Meaning
`page`	yes	yes	original page number
`status`	yes	yes	`ok` or `error`
`attempts`	yes	yes	number of attempts used
`text`	yes	no	extracted OCR text
`error_kind`	no	yes	structured failure class
`error_message`	no	yes	diagnostic message
`http_status`	no	optional	HTTP status where applicable

Table 5: `pdfocr` page result schema

- `src/chunkvec/runtime_config.nim`: shared CLI and runtime configuration;
- `src/chunkvec/input_chunks.nim`: `<chunk ...>` parser;
- `src/chunkvec/chunk_store.nim`: SQLite schema, vector extension, insert and search;
- `src/chunkvec/embeddings_client.nim`: embedding request construction and retrying query embedding;
- `src/chunkvec/pipeline.nim`: bounded concurrent embedding ingest;
- `src/chunkvec/sqlite_vector_paths.nim`: platform-specific extension resolution;
- request-id, retry queue, retry/error, constants, types, and logging modules; and
- tests for parsing, configuration, request ids, embeddings client, and SQLite/vector integration.

The repository exposes two separate commands because storage and search have different operational profiles.

5.4.2 `chunkvec`: Chunk Parser

The chunk parser expects every chunk to start at a line with a `<chunk ...>` marker. Supported attributes are:

- `page=N`, where `N` is an integer;
- `label="..."`, where the value is double-quoted.

Unknown attributes are rejected. Empty chunk bodies are rejected. Whitespace before the first marker is ignored, and chunk bodies are trimmed. The parser searches for markers at line starts so that literal occurrences of `<chunk` inside content do not automatically split the document.

This strict input grammar matters because retrieval metadata becomes part of the persistent database. Permissive parsing would make stored retrieval state harder to reason about.

5.4.3 `chunkvec`: Database and Vector Search

The SQLite schema stores:

- `source`;
- `text`;
- `embedding` as a BLOB;
- `doc_id`;
- `kind`;
- `page`;
- `label`;
- `created_at`.

There is an index over `(doc_id, kind, page)` to support common filters. The vector extension is loaded explicitly, and `vector_init` initialises the vector column with fixed dimension and distance options. After insertions, `vector_quantize` and `vector_quantize_preload` prepare the quantized vector search path.

The ingest command runs inside a transaction. It also supports resumability by selecting already stored chunks with the same `source`, `doc_id`, `kind`, page, label, and text, then deleting those chunks from the pending ingest list. This makes repeated ingest idempotent for unchanged chunks.

Search has two paths:

- unfiltered search: run a quantized vector scan and order by distance then row id;
- filtered search: join vector scan results with metadata filters, then order and limit.

The label filter is normalised by lowercasing and removing underscores, matching the tool definition's stable-label policy.

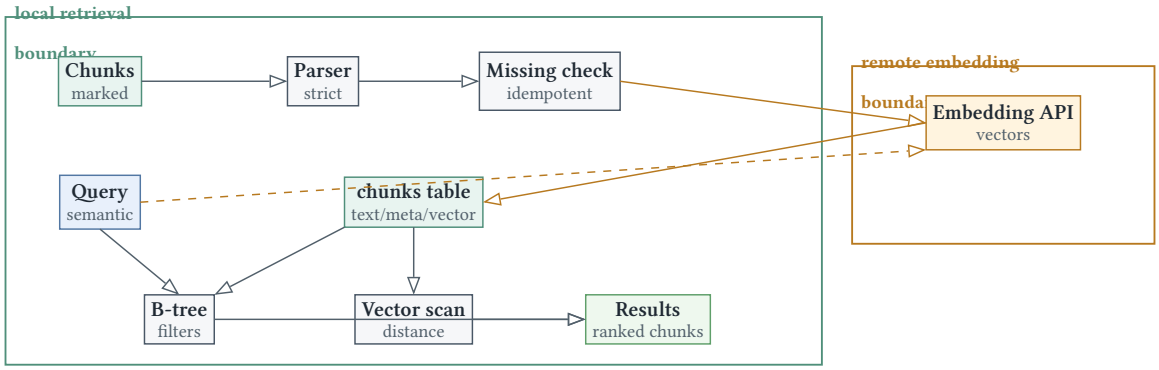

Figure 7: `chunkvec` local storage, metadata filtering, vector scan, and remote embedding boundary.

Figure 7 shows the persisted retrieval model. The implementation stores vectors in the same logical row as text and metadata, then uses `sqlite-vector` scanning functions over the embedding column.

5.4.4 `chunkvec`: Embedding Pipeline

The ingest pipeline uses the same state-machine pattern as OCR, but its terminal action is database insertion rather than JSONL emission. On each successful embedding response, the pipeline verifies:

1. the response parses as an embedding result;
2. the response contains an embedding;
3. the embedding length equals the configured dimension; and
4. the row can be inserted through the prepared statement.

Failures mark the chunk as unsuccessful. Successful rows are committed when the transaction completes. If all chunks are already present, the command exits successfully without sending remote embedding requests.

Figure 8 shows the actual ingest sequence implemented by `cvstore.nim`, `chunk_store.nim`, and `pipeline.nim`.

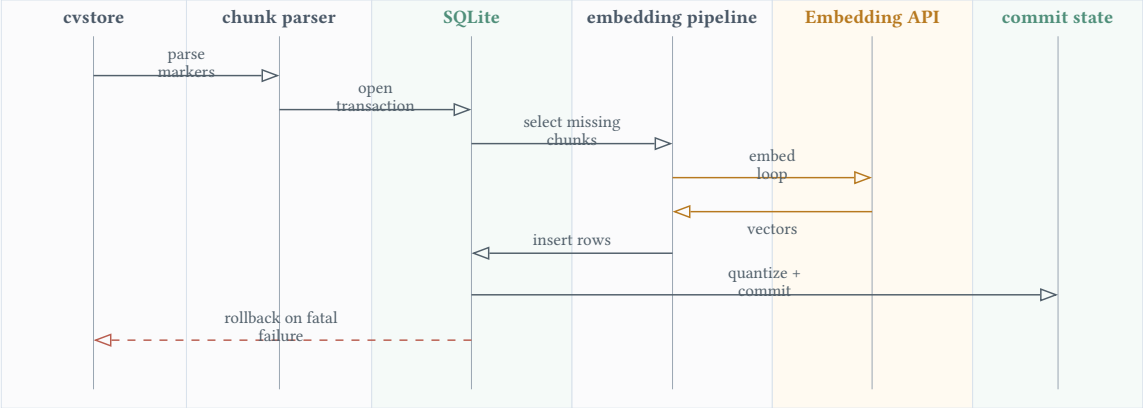

Figure 8: `cvstore` ingest sequence showing strict parsing, missing-chunk detection, embedding requests, and transaction finalisation.

5.5 TTS Implementation

5.5.1 `chunktts`: Module Structure

`chunktts` implements ordered text-to-speech. Its structure contains:

- `src/app.nim`: application entry point;
- `src/chunktts/runtime_config.nim`: CLI and configuration;
- `src/chunktts/chunk_split.nim`: marker-based chunk splitting;
- `src/chunktts/tts_client.nim`: speech request construction;
- `src/chunktts/sndfile_wrap.nim`: memory decoding, audio validation, `.opus` writing;
- `src/chunktts/pipeline.nim`: bounded concurrent TTS pipeline;
- request-id, retry queue, retry/error, constants, types, and logging modules; and
- tests for chunk splitting, speech requests, audio wrapper, retry mapping, request ids, and pipeline integration.

The command interface is intentionally small:

```
chunktts INPUT.txt OUTPUT.opus
```

All chunking decisions are encoded in the input file through `<bk>` markers.

5.5.2 `chunktts`: Speech and Audio Algorithms

The chunk splitter is simple by design: split on the configured marker, trim whitespace, and discard empty chunks. More complex linguistic chunking belongs to `tts-tool`, not to the executable.

The TTS request builder creates an OpenAI-compatible audio speech request with:

- model;
- input text;
- voice;
- WAV response format; and
- speed.

WAV is requested because the pipeline validates decoded audio before writing the final `.opus` file. `sndfile_wrap` implements virtual I/O over returned bytes, decodes them into float samples, checks sample rate/channel consistency across chunks, and writes the final Ogg Opus file.

The pipeline stores decoded chunks by sequence id. If every chunk succeeds, the output writer concatenates them in input order. If any chunk fails permanently, the final output file is not written.

The integration test uses a local asynchronous HTTP server to verify:

- actual bounded concurrency (`maxActive == 2` in the test);
- retry behaviour for a simulated HTTP 429 response;
- output file existence only after success; and
- final audio properties such as sample rate, channel count, and frame count.

Figure 9 shows the audio publication gate. The key implementation detail is that returned audio is decoded into sample buffers before final `.opus` writing; this is why the tool can validate format consistency across chunks.

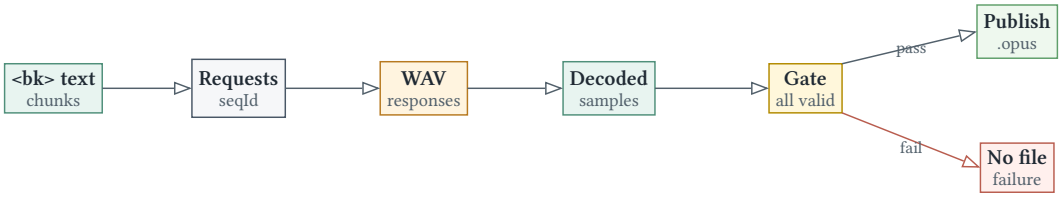

Figure 9: `chunktts` publication gate: only complete and format-consistent decoded chunks produce the final `.opus` file.

5.6 Shared Infrastructure

5.6.1 `relay`: HTTP Transport Library

`relay` abstracts libcurl multi behind a Nim API. Its important public types are:

- `RequestSpec`;
- `RequestBatch`;
- `RequestResult`;
- `Response`;
- `TransportError`; and
- `Relay`.

The client has a worker thread. The creating thread submits requests and drains ready results. The worker thread owns the libcurl multi handle, dispatches queued requests into available easy handles, drives `curl_multi_perform` and `curl_multi_poll`, reads completion messages, and pushes completed results into a ready-results queue.

The internal queues are:

- `queue`: pending request wraps;
- `inFlight`: map from easy-handle pointer to request wrap;
- `availableEasy`: reusable easy handles; and
- `readyResults`: completed request results.

The lifecycle has two shutdown modes:

- `close`: finish queued/in-flight work and join the worker;

- `abort`: cancel queued/in-flight work, return cancellation results, and join promptly.

The transport error classifier maps curl errors into timeout, DNS, TLS, cancellation, protocol, network, and internal categories. This abstraction is what lets the core tools implement consistent retry policies. The implementation preserves the concurrency boundary shown in Figure 3: the caller owns scheduling, ordering, retries, and publication state, while `relay` owns libcurl multiplexing and completion delivery.

5.6.2 `jsonx`: JSON Library

`jsonx` provides a JSON lexer/parser, object mapping, streaming writers, and raw JSON support. The library is used in two ways:

- generic typed serialisation/deserialisation for configuration and API schemas;
- custom writers for output schemas whose fields depend on state.

The library supports `RawJson` and `CanonRawJson`. `RawJson` preserves arbitrary JSON fragments; `CanonRawJson` normalises object fields by sorting keys and keeping the final value for duplicate keys. This is useful for schemas and caching scenarios where an application needs to carry provider-defined JSON without modelling every field as a Nim type.

For this project, the most important design property is that JSON output is centralised in type-specific writers. `PageResult`, for example, emits common fields and then emits either `text` or error fields depending on status. This prevents scattered string concatenation and reduces schema drift.

5.6.3 `openai`: API Schema Library

The `openai` repository is a thin typed SDK that stays out of the transport layer. It depends on `relay` for HTTP and `jsonx` for JSON. The inspected modules cover:

- chat completions;
- embeddings;
- audio speech;

- request construction helpers;
- retry helpers; and
- schema modules for request and response shapes.

The chat layer provides helpers for:

- text messages;
- multimodal message parts;
- image URLs;
- tool definitions;
- structured response formats; and
- response accessors such as `firstText`.

The embeddings layer creates embedding requests and validates parsed embedding content as float or base64. The audio speech layer creates speech requests with model, input, voice, response format, speed, and optional provider-specific body fields.

The important design choice is transport transparency. `openai` builds `RequestSpec` values or appends them to `RequestBatch`; it does not execute them directly. This lets `pdfocr`, `chunkvec`, and `chunktts` control batching, timeouts, retry timing, and shutdown.

5.6.4 Configuration and Secrets

All core tools follow the same configuration pattern:

1. Load built-in defaults.
2. Read optional `config.json` from the executable directory.
3. Resolve the API key from `DEEPINFRA_API_KEY` when present.
4. Normalise values into safe ranges.

Examples of normalisation include positive concurrency limits, positive timeouts, non-negative retry counts, WebP quality constrained to `[0, 100]`, and TTS speed constrained to `[0.25, 4.0]`.

The OCR subsystem uses `allenai/olmOCR-2-7B-1025` through an OpenAI-compatible multimodal endpoint. The RAG subsystem uses an OpenAI-compatible embeddings endpoint, with `Qwen/Qwen3-Embedding-0.6B` documented as the default embedding model in `chunkvec`. The TTS subsystem uses an OpenAI-compatible audio speech endpoint, with `hexgrad/Kokoro-82M` documented as the default speech model in `chunktts`.

The runtime does not inspect arbitrary shell profiles or filesystem locations to discover secrets. The credential boundary is explicit: environment variable or config file.

5.6.5 Cross-Repository Data Contracts

The repositories communicate through simple artefacts. This artefact-based design allows the system to be inspected at each stage. It also supports standalone use: every core processing step can be invoked without the agent.

Figure 10 summarises the executable artefact chain. Unlike the global architecture diagram, this view focuses on concrete files and streams that can be inspected by a user or test.

5.7 Implementation Outcomes

The implementation shows a consistent system design:

- instruction repositories define safe agent behaviour;
- core tools implement deterministic processing contracts;
- shared libraries factor out HTTP, JSON, and model API handling;
- bounded concurrency is implemented through `relay`;

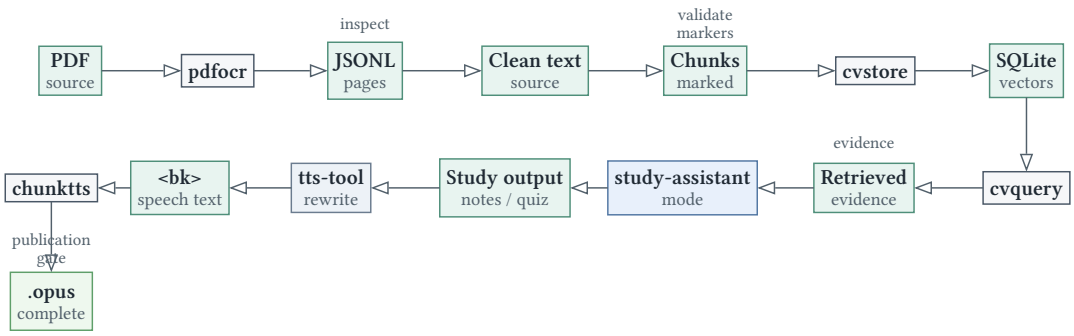

Figure 10: Artefact lineage across OCR, retrieval, study generation, and optional audio publication.

- result ordering is maintained through sequence ids and staging;
- persistent retrieval is implemented through SQLite plus vector search; and
- speech output is validated before final artefact publication.

The implementation is therefore not merely a collection of scripts. It is a modular study-assistant architecture with explicit contracts at every boundary.

6 Verification, Testing, and Evaluation

6.1 Evaluation Methodology

The evaluation strategy reflects the system's mixed nature. Some properties are deterministic software properties and can be verified with unit or integration tests. Other properties depend on live remote models and are evaluated through recorded empirical runs. The methodology therefore separates:

- **contract verification:** CLI, parsing, schema, ordering, retry, database, and audio invariants;
- **pipeline verification:** bounded concurrency and successful artefact production under controlled or recorded conditions;
- **model evaluation:** OCR quality and cost on a fixed benchmark; and
- **operational evaluation:** throughput, memory, exit codes, and failure semantics.

This separation prevents model variability from obscuring local implementation correctness.

6.2 Verification Evidence Topology

Figure 11 maps repositories and evidence types to the evaluation claims they support.

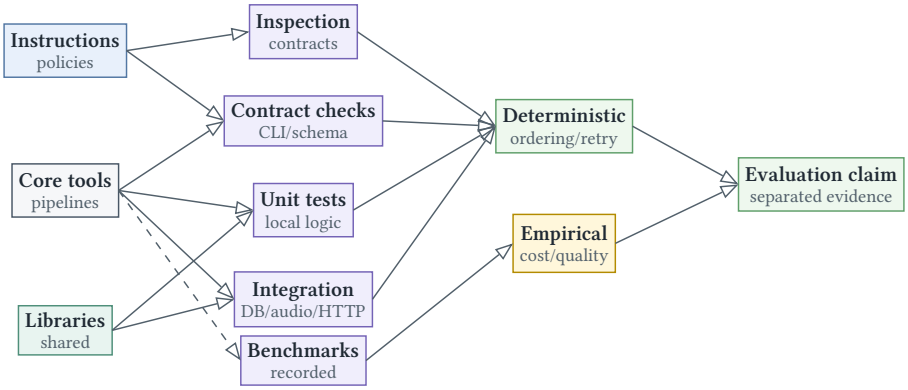

Figure 11: Verification evidence topology separating deterministic implementation evidence from empirical model and performance evidence.

The instruction repositories do not have conventional unit tests because their primary artefacts are operational rules. They are validated by checking whether the rules map cleanly onto the executable contracts and whether they preserve source-grounding constraints.

6.3 OCR Contract Verification

The OCR contract has the following properties:

- input is one PDF and exactly one page selector mode;
- page selection is sorted and duplicate-free after normalisation;
- stdout contains JSONL page records only;
- stderr contains logs and diagnostics;
- each selected page produces one page result on non-fatal completion;
- page-result order matches the normalised selection order;
- each result contains attempt count and status;
- error records have structured error kinds; and
- fatal startup/runtime failures produce exit code `3`.

The deterministic tests cover page selection, request-id packing, retry queue behaviour, retry/error mapping, and JSON result writing. Ordering is primarily enforced by construction: the pipeline emits only from `staged[nextEmitSeqId]` and increments that pointer monotonically.

6.4 RAG Contract Verification

The RAG contract has storage and search components.

For storage:

- every chunk begins with a valid `<chunk ...>` marker;
- unknown marker attributes are rejected;
- empty chunk bodies are rejected;

- `doc` and `kind` are command-level metadata, not marker attributes;
- embeddings must match the configured dimension;
- database inserts occur inside a transaction; and
- already-stored chunks can be skipped.

For search:

- the query is embedded once;
- the vector extension is loaded explicitly;
- the vector column is initialised with a fixed dimension;
- filters are applied through SQL parameters;
- results are ordered by vector distance and row id; and
- rendered output includes enough metadata for inspection.

The SQLite/vector integration tests are especially important because they verify that the database schema, vector extension, insertion, and search path work together.

6.5 TTS Contract Verification

The TTS contract is artefact-oriented:

- input is one text file and one output path;
- chunks are produced by splitting on `<bk>`;
- empty chunks are dropped;
- speech requests are bounded by `max_inflight`;
- transient failures are retried within the attempt bound;
- returned audio bytes are decoded and validated;
- sample rate and channel count must match across chunks; and
- the final `.opus` file is written only if all chunks succeed.

The local integration test uses a controlled HTTP server that simulates concurrency and retry pressure. It verifies that the pipeline reaches the expected maximum active request

count, retries a chunk after an HTTP 429 response, and writes a valid output file with expected sample rate, channel count, and frame count.

6.6 Shared Library Verification

`relay` is verified through tests for lifecycle contracts, request-body round trips, header parsing, single-request helpers, batch helpers, and ordering behaviour. This matters because all remote-model pipelines depend on relay's guarantee that request ids and result bodies are returned intact.

`jsonx` is verified through parser and compliance tests. The project relies on `jsonx` for configuration, API payloads, and output records, so JSON parser correctness is a cross-cutting reliability requirement.

`openai` is verified through tests for chat, embeddings, audio speech, and retry helpers. These tests validate request/response schema compatibility independent of the higher-level pipelines.

6.7 OCR Throughput Evaluation

The main throughput benchmark uses a fixed 72-page slide PDF. Figure 12 reports the recorded result visually; exact values are repeated in Appendix Appendix C.

The speedup is:

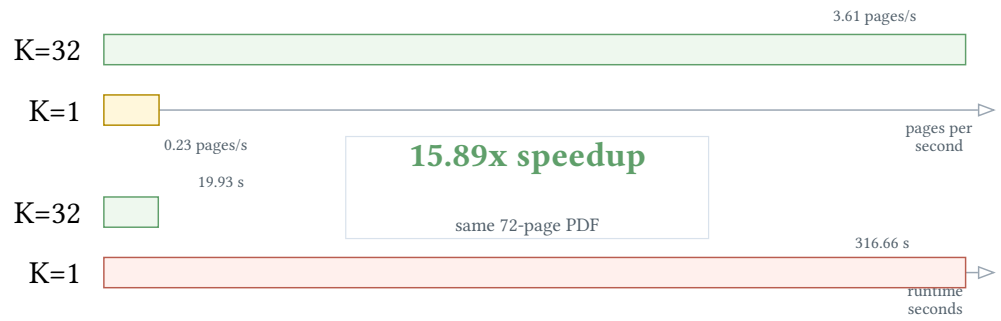

Figure 12: OCR throughput benchmark showing runtime reduction and throughput increase under bounded concurrency.

$$S = \frac{316.66}{19.93} = 15.89 \tag{6}$$

The absolute reduction is:

$$316.66 - 19.93 = 296.73 \text{ s} \tag{7}$$

The relative reduction is:

$$\frac{296.73}{316.66} \times 100 = 93.71\% \tag{8}$$

The result is consistent with the design. OCR requests are network-bound, so bounded concurrency overlaps provider latency while the main thread preserves ordering.

6.8 OCR Model Evaluation

The 68-page scientific OCR benchmark uses locked human gold labels. The benchmark includes pages from 34 source PDFs and covers equations, tables, diagrams, and multi-column layouts. All evaluated models completed 68/68 pages.

Figure 13 summarises the model-selection trade-off. Exact CER, WER, reading-order, math, and cost values are reported in Appendix Appendix C. Short model names correspond to `PaddlePaddle/PaddleOCR-VL-0.9B`, `allenai/olmOCR-2-7B-1025`, and `deepseek-ai/DeepSeek-OCR`.

The recorded interpretation is:

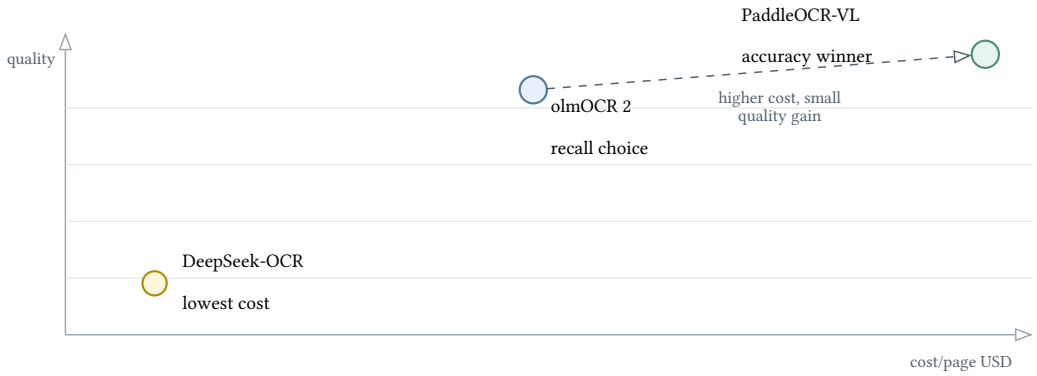

Figure 13: OCR model cost/quality trade-off used to justify the selected OCR model.

- `PaddlePaddle/PaddleOCR-VL-0.9B` gives the strongest strict-accuracy aggregate in this run.
- `allenai/olmOCR-2-7B-1025` gives the strongest recall-oriented aggregate.
- `deepseek-ai/DeepSeek-OCR` gives the lowest cost and strongest cost-efficiency-weighted score.

The selected OCR model, `allenai/olmOCR-2-7B-1025`, is justified by this trade-off. PaddleOCR-VL gives slightly stronger strict accuracy, but at a higher recorded cost; DeepSeek-OCR is cheapest, but its accuracy and structure metrics are substantially weaker. olmOCR 2 is therefore the most suitable choice for this project because it provides the strongest recall-oriented result while keeping cost below the strict-accuracy winner. In a study-assistant workflow, recall is especially important: a missing theorem, definition, or equation can produce a defective study artefact even when the remaining transcription has acceptable CER or WER.

6.9 RAG Evaluation Criteria

The RAG subsystem is not evaluated by question-answering accuracy in this report because its core implementation responsibility is retrieval infrastructure. The relevant evaluation criteria are:

- correctness of chunk parsing;
- preservation of source text in chunk bodies;
- metadata stability;
- embedding dimensionality validation;
- database persistence;
- search filter correctness;
- deterministic result ordering; and
- local search after ingest.

These criteria align with the system's role: `chunkvec` is a retrieval substrate, while `study-assistant` is the component that uses retrieved passages for generation.

6.10 TTS Evaluation Criteria

The TTS subsystem is evaluated as an artefact pipeline. The main criteria are:

- successful transformation from marked text to ordered audio chunks;
- retry robustness for transient API failures;
- audio decodability;
- consistent audio format across chunks;
- correct `.opus` publication; and
- absence of partial final artefacts on failure.

Subjective naturalness is not evaluated directly because the project does not train or compare TTS models. The implementation assumes the provider model is responsible for acoustic quality and focuses on reliable preparation and assembly.

6.11 Reliability Evaluation

The three model-calling tools share reliability mechanisms:

- bounded in-flight request limits;
- maximum attempt counts;
- retry queues ordered by due time;
- exponential backoff and jitter through shared retry helpers;
- transport and HTTP classification;
- abortive shutdown on fatal errors; and
- stable exit codes.

Figure 14 summarises failure semantics.

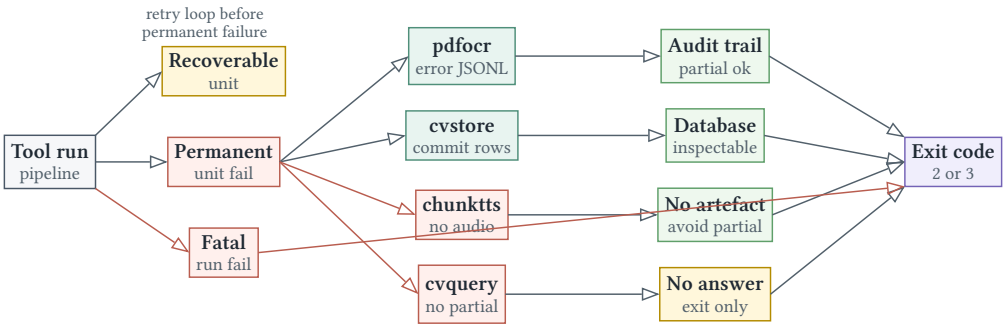

Figure 14: Failure semantics as branching publication behaviour across recoverable, permanent, and fatal failures.

These semantics are tailored to artefact type. Partial OCR can still be useful and auditable. Partial final speech audio is not published because it would appear complete while silently omitting content.

6.12 Performance and Memory Interpretation

The benchmark evidence supports the claim that bounded concurrency improves throughput for OCR. It does not establish a universal speed guarantee. Performance depends on:

- provider latency;
- rate limits;
- local PDF rendering speed;
- WebP encoding speed;
- input page complexity;
- network conditions; and
- model response length.

Memory interpretation also requires care. Peak RSS is the appropriate process-level metric for comparing memory in recorded runs. Allocator-internal counters are useful diagnostics but are not equivalent to process peak memory.

6.13 Threats to Validity

The main threats to validity are:

- **Provider variability:** remote model latency and behaviour can vary by time and deployment.
- **Dataset specificity:** the recorded OCR benchmark may not represent all study documents.
- **Single-run measurements:** recorded throughput runs do not report statistical variance.
- **Model drift:** hosted models and pricing can change.
- **No subjective TTS evaluation:** the project verifies artefact correctness, not listener preference.
- **No end-to-end learning-outcome study:** the report evaluates system behaviour, not educational outcomes with human participants.

These limitations do not invalidate the implementation results, but they define the scope of claims that can be made rigorously.

6.14 Technical Evaluation Findings

The technical verification evidence supports the system architecture. Local deterministic tests cover the core invariants. The TTS integration test demonstrates concurrency, retry, and audio finalisation under controlled conditions. The OCR throughput benchmark demonstrates the practical benefit of bounded concurrency. The scientific OCR benchmark provides model-level evidence for olmOCR 2 and situates it among measured alternatives. The following end-to-end workflow evaluation examines how these technical properties appear in student-facing use.

7 End-to-End Workflow Evaluation

The workflows are evaluated against the objectives defined by the `study-assistant` and supporting tool instructions: outputs should be grounded in prepared source material, appropriate to the selected study mode, useful for exam preparation, and operationally reproducible enough for repeated use. The assessment therefore distinguishes serious errors from acceptable transformations required by the workflow. A flashcard deck, for example, is not expected to preserve every slide verbatim; a transcription or RAG source ingest is held to a stricter fidelity standard.

7.1 WF1: OCR-Grounded Essay Practice

Prompt.

> Use the study-assistant in essay mode on the lecture slides at `08_Association_Analysis.pdf` for exam preparation

The recorded output is an essay-mode workflow rather than a strict transcription workflow. Under the `study-assistant` contract, essay mode should produce three to four exam-style prompts, include conceptual and applied questions, provide sample answers of roughly 200 words, and remain grounded in the prepared source text. Judged against that objective, the workflow is largely successful.

The OCR stage extracted the Association Analysis slides into structured Markdown. Key concepts and formulas were preserved sufficiently for downstream generation, including transaction tables, support, confidence, Apriori, lift, and subjective interestingness. Representative OCR evidence includes:

$$s = \frac{\sigma(\text{Milk, Diaper, Beer})}{|T|} = \frac{2}{5} = 0.4 \tag{9}$$

$$c = \frac{\sigma(\text{Milk, Diaper, Beer})}{\sigma(\text{Milk, Diaper})} = \frac{2}{3} = 0.67 \tag{10}$$

The generated essay set covered the main examinable ideas in the lecture: Apriori pruning, support and confidence, lift, association-rule generation, and interestingness measures. A representative prompt and answer excerpt were:

> Discuss the Apriori principle and explain how it reduces the computational cost of frequent itemset mining.
> The Apriori principle states that if an itemset is frequent, then all of its subsets must also be frequent. Equivalently, if an itemset is infrequent, all of its supersets must be infrequent. This follows from the anti-monotone property of support…

This is a strong match to the essay-mode objective. The answer asks for explanation rather than recall alone, uses formal academic verbs, and connects the principle to computational pruning. The Tea-Coffee answer is also well-grounded: it uses the source example where confidence is `150/200 = 0.75` but the base coffee probability is `800/1000 = 0.80`, leading to lift `0.9375` and a slight negative association.

The limitations are modest relative to the task. OCR introduced small local errors, such as truncating “Make offers, personalize” to “Make offers, persona”. Diagram interpretation also produced occasional awkward wording in the Apriori lattice explanation. These issues would matter more for a verbatim transcription than for essay practice. For the recorded objective, they did not materially prevent correct exam-style generation, but they indicate that transcription-quality outputs should be retained separately when later workflows require exact provenance.

7.2 WF2: RAG-Based Exam Revision

Storage prompt.

> Use the OCR tool to extract pages 358-402 from Introduction to Data Mining, then use the RAG tool to store the extracted text so I can search it later for exam revision

Search prompt.

> Use the RAG tool to search my stored notes for how does the Naive Bayes classifier handle zero probability conditions

The intended RAG workflow is to convert source material into coherent chunks and retrieve relevant source content later. The store output reports successful OCR extraction, cleaning, and storage of the textbook section as 26 semantic chunks. The chunking is broadly appropriate for exam revision: chunks are organised around topics such as nearest-neighbour classification, Bayes' theorem, the Naive Bayes assumption, zero conditional probability, Bayesian networks, and inference.

The relevant stored chunk is directly aligned with the later query:

> `<chunk page=379 label="Zero Conditional Probability">`
>
> Handling Zero Conditional Probabilities: If the conditional probability for any attribute is zero, the entire class-conditional probability becomes zero. This happens when the number of training instances is small and the number of possible attribute values is large.
>
> Two alternate estimates to avoid vanishing probabilities:
>
> Laplace estimate: `P(Xi=c|y) = (nc + 1) / (n + v) ...`
>
> m-estimate: `P(Xi=c|y) = (nc + m*p) / (n + m) ...`

The search result answered the query accurately:

> If the conditional probability for any attribute is zero, the entire class-conditional probability becomes zero.

Example (Loan Default Prediction): For class Yes, no training instance has Marital Status=Married, so `P(Status=Married|Yes) = 0`, which zeroes out the entire product `P(x|Yes)`.

Two alternate estimates to avoid vanishing probabilities:

- Laplace estimate: `P(Xi=c|y) = (nc + 1) / (n + v)`
- m-estimate: `P(Xi=c|y) = (nc + m.p) / (n + m)`

This demonstrates the value of RAG for the study-assistant objective. The answer is not only generally correct; it is grounded in the course-specific loan-default example and retrieves both smoothing methods discussed in the source. Compared with a naive prompt about Naive Bayes, the RAG workflow materially improves alignment with the assigned material.

The main limitation is not answer quality but traceability. The `rag-tool` search instructions prefer verbatim returned chunks, while the recorded answer is a concise synthesis. For a student, this synthesis is useful and sufficient for revision. For strict evaluation, it weakens the ability to separate retrieved evidence from generated wording. A future version should record the retrieved chunks alongside the final answer rather than replacing the answer with raw chunks only.

Preprocessing quality is adequate for the demonstrated query. Some OCR formula fragments in the broader Naive Bayes section are malformed, but the chunk used for zero-probability retrieval preserves the essential explanation and estimates. The extra ingest artefact spanning pages 402-458 creates some provenance ambiguity, but it did not affect the observed search result. The appropriate conclusion is that WF2 is sufficient for targeted exam revision, while stronger metadata and retrieval logs are needed for rigorous reproducibility.

7.3 WF3: Flashcard Generation

Prompt.

> Use the study-assistant in flashcard mode on the lecture slides at `07_Anomaly_Detection.pdf` for exam revision

The flashcard mode requires a two-column Markdown table of high-value terms, definitions, formulas, distinctions, and core concepts. The generated deck contains 25 cards and satisfies this objective well. It covers anomaly definitions, causes, point/contextual/collective anomalies, supervised, semi-supervised and unsupervised detection, label versus score outputs, statistical tests, proximity-based methods, density-based methods, LOF, cluster-based detection, real-world issues, and applications.

Representative output:

Front (Term/Question)	Back (Definition/Answer)
What is an anomaly (outlier)?	An object that is different from most other objects in the dataset.
What are the three main causes of anomalies?	Data from a different class/mechanism; natural variation; measurement/collection errors.
Contextual anomaly	An instance that is anomalous only within a specific context. Requires a notion of context.
Label vs. Score output in anomaly detection	**Label:** each instance is tagged normal or anomaly. **Score:** each instance gets a numeric anomaly score allowing ranking; requires an additional threshold.
LOF (Local Outlier Factor)	A relative density score comparing the average density of k neighbours with the density of the point.

Table 6: Representative anomaly-detection flashcards

The cards are factually consistent with the lecture and suitable for active recall. They are concise without being merely keyword-based, and they include both definitions and method limitations. This aligns with the prompt rule to include high-value distinctions rather than duplicate low-level details.

The output is less suitable for calculation practice or deep derivations, but that is not the primary purpose of flashcard mode. A small number of cards compress technical content, such as LOF and Grubbs' test, into revision-level answers. This is acceptable for flashcards, provided they are not presented as a replacement for full notes or worked examples.

A second flashcard artefact for Association Analysis shows similar strengths. It covers support count, support, confidence, Apriori, FP-growth, rule generation, lift, contingency tables, and interestingness measures. One formula preserves the source notation for lift:

```
Lift(X -> Y) = c(X -> Y) / sigma(Y)
```

Because the slides elsewhere use `sigma` for support count, this notation is potentially ambiguous. The issue is inherited from the source representation rather than invented by the model. For the study objective, the surrounding cards and examples make the concept understandable, but a validation pass could improve clarity by rewriting the denominator as `support(Y)`.

Overall, WF3 is sufficient for the study-assistant objective of exam-ready active recall. The main improvement would be automated checks for duplicate cards, formula ambiguity, and coverage against slide headings.

7.4 WF4: Study Notes and Text-to-Speech

Prompt.

> Use the study-assistant in study-notes mode on the lecture slides at `06_Clustering_Density_Validation.pdf`, then use the TTS tool to convert the notes to audio so I can listen while revising

This workflow combines two objectives: study-note generation and speech preparation. The notes should prioritise understanding, relationships, and exam relevance; the TTS input should remove Markdown and mathematical syntax that would sound unnatural when spoken.

The generated notes cover the central lecture content: DBSCAN, core/border/noise points, algorithm steps, DBSCAN strengths and limitations, cluster validation, validity indices, similarity-matrix inspection, SSE, cohesion, separation, entropy, and purity. Representative study-note output:

> DBSCAN groups points by local density rather than by distance to centroids or by merging clusters hierarchically. Two parameters control the algorithm:
>
> - Eps: the radius that defines the neighbourhood around a point.
> - MinPts: the minimum number of points that must fall within the Eps-radius for a point to qualify as dense.

The TTS rewrite is a meaningful modality transformation rather than a direct file conversion. It rewrites headings, bullets, formulas, and table-like content into spoken prose:

> DBSCAN, Density-Based Spatial Clustering. DBSCAN groups points by local density rather than by distance to centroids or by merging clusters hierarchically. Two parameters control the algorithm. Eps, written epsilon, is the radius that defines the neighbourhood around a point.

For mathematical content, the rewrite is appropriate for listening:

```
p sub i j equals m sub i j divided by m sub j
```

This satisfies the TTS objective. The speech input is coherent, listenable, and avoids raw Markdown or LaTeX. It is also chunked into spoken units with `<bk>` markers, which supports the downstream audio tool.

There are minor technical shifts. The notes use “at least MinPts” for a core point, while the slide text says “more than MinPts”; the generated version is consistent with common DBSCAN terminology, but it is not a verbatim rendering of the local slide wording. The entropy formula is also normalised to the standard negative-sum form after OCR recorded it without the negative sign. For study notes, these choices are defensible because they improve conceptual correctness. For strict source auditing, they should be marked as normalisations.

WF4 is sufficient for passive revision. It should not be treated as a replacement for visual slide study, because examples, plots, and table details are necessarily condensed for audio. The operational record is also positive: OCR used a cache hit, `chunktts` completed, and no stderr failures were recorded.

7.5 Cross-Workflow Assessment

The recorded workflows meet the practical objectives of the study-assistant: they transform course PDFs into exam-oriented essays, searchable source notes, flashcards, written notes, and speech-ready revision material. The outputs are mostly grounded in the original course material and are pedagogically useful for the intended modes.

The limitations are best understood as engineering improvements rather than workflow failures:

- source-preserving artefacts should be retained when later stages depend on exact provenance;
- RAG should record retrieved chunks and final synthesis together;

- OCR cleanup should flag formula and diagram uncertainty instead of silently relying on manual judgement;
- generated flashcards and notes would benefit from lightweight validation for coverage, duplication, and formula ambiguity;
- run manifests should link source PDFs, OCR cache entries, generated text, TTS input, and final audio.

The stderr logs for the recorded workflows are empty, and inspected OCR cache entries report successful page extraction with `status="ok"` and `attempts=1`. This supports the conclusion that the recorded runs were operationally stable. It does not prove robustness under all provider, rate-limit, or document conditions, but it is sufficient evidence that the current implementation can complete the demonstrated study workflows.

8 Discussion

8.1 Contribution Summary

The project contributes a complete agent-based study assistant architecture rather than a single isolated algorithm. Its contribution is the design and evaluation of a system that transforms educational source material into study-ready outputs through coordinated OCR, retrieval-augmented assistance, and text-to-speech.

The first contribution is conceptual. The project frames study assistance as a source-grounded workflow. A student begins with course material that may be scanned, fragmented, lengthy, or unsuitable for direct prompting. The system supports a sequence of learning-oriented transformations: extraction, cleaning, retrieval, explanation, active-recall generation, essay practice, and audio revision. This framing is academically relevant because it connects AI assistance to realistic student study activity rather than treating generation of summaries as the central problem.

The second contribution is architectural. `study-assistant` acts as an agent-level coordinator that maps user intent to explicit study modes such as transcription, study notes, lecture-style explanation, simplified explanation, flashcards, mind maps, quizzes, essay practice, retrieval, and audio preparation. The agent does not absorb all processing into one conversational prompt. It coordinates specialised tools with distinct responsibilities, which gives the system clearer boundaries than a monolithic assistant.

The third contribution is technical. The project implements a modular tool ecosystem for OCR, RAG, and TTS. The OCR component addresses input readiness by converting scanned or PDF-based material into text. The RAG component addresses source-grounded access by storing and searching prepared study material. The TTS component addresses output modality

by converting prepared text into audio suitable for listening-based revision. Each component is usable within the agent workflow and independently as a focused processing tool.

The fourth contribution is infrastructural. The Nim-based implementation is supported by custom libraries for HTTP transport, JSON handling, and OpenAI-compatible API interaction. `relay`, `jsonx`, and `openai` provide shared foundations used across the processing tools. This reduces duplication, makes model-service interaction explicit, and provides reusable components for similar systems.

8.2 Interpretation of Findings

The system-level contribution lies in the integration of multiple AI modalities into a unified study assistant while preserving separation of concerns. OCR, retrieval, language generation, and speech synthesis are often treated as separate capabilities. In this project they are organised around the lifecycle of study material:

1. material becomes text through OCR when required;
2. text becomes reusable knowledge through semantic storage;
3. retrieved passages become explanations, notes, questions, or other study artefacts;
4. selected text becomes audio for listening-based revision.

The value of the assistant emerges from this coordination. OCR alone does not produce revision practice. Retrieval alone does not prepare scanned material. Text-to-speech alone does not decide what should be spoken. The agent-based design connects these stages while keeping their responsibilities distinct.

This separation between agent orchestration and standalone tool functionality is an important design decision. The agent defines the study workflow from the user's perspective. The tools define stable processing surfaces. This makes the system easier to test and reason about because many behaviours can be evaluated through tool contracts, schemas, stored artefacts, and pipeline invariants rather than through informal conversational behaviour alone.

8.3 Practical Implications

The project is practically relevant because it supports common student learning workflows. It helps students work with material that is difficult to use directly, such as scanned pages or long notes. It supports retrieval over prepared sources, which is useful when a student needs to locate a concept or explanation without manually scanning a document. It supports active-recall artefacts such as flashcards and quizzes, which align with established study practices. It also supports audio revision, which can make study more flexible and accessible.

The system does not assume a single learning output. A student checking extracted material needs faithful transcription. A student approaching a difficult topic may need a lecture-style explanation or a simpler explanation. A student preparing for an examination may need flashcards, quizzes, or essay prompts. A student revising away from a desk may need audio. Representing these as related modes within one assistant is the practical value of the architecture.

The architecture is reusable beyond the immediate project. Any educational workflow that requires document preparation, semantic retrieval, controlled generation, and alternative output modalities can reuse the same pattern. This gives the system relevance as a framework for future educational tools and as a case study in applied AI system design.

8.4 Open-Source Value

The open-source nature of the project is significant for an academic computing artefact. Open-source release allows the system to be inspected, reproduced, modified, and extended. This is particularly important for AI-assisted education, where reliability, data handling, source grounding, and tool behaviour should be open to scrutiny.

For developers, the project provides reusable components for building similar systems. The OCR, RAG, and TTS tools can be adapted independently. The supporting libraries can be

reused in Nim applications that require HTTP transport, JSON handling, or OpenAI-compatible model access. The agent/tool separation provides a pattern for systems in which an instruction-driven layer coordinates deterministic processing tools.

For researchers and students, the system provides an inspectable case study in applied AI architecture. It demonstrates how model services can be placed inside a controlled software system rather than treated as isolated prompts. It also provides a basis for further experimentation, such as alternative OCR models, different embedding models, richer retrieval evaluation, additional study modes, accessibility-focused workflows, or local model deployment.

Open-source release does not by itself establish educational impact. Its value is that the claims made in the report can be related directly to inspectable source artefacts, and that the architecture can be reused or challenged by others.

8.5 Limitations

The project should be interpreted as a system-design and engineering contribution, not as a complete pedagogical intervention study. It does not establish that use of the system improves grades, retention, or long-term learning outcomes. The evaluation demonstrates tool reliability, throughput, ordering behaviour, and model suitability within the scope of the implemented system.

The system also depends on model services for OCR, embeddings, language generation, and speech synthesis. This makes the architecture practical and modular, but it means that quality, latency, cost, and availability remain partly dependent on external providers. The contribution is therefore the orchestration, tool design, and processing architecture around these services rather than the training of new foundation models.

Finally, the system is intentionally scoped around study preparation and revision. It does not replace instructors, textbooks, formal feedback, or assessment. Its role is to help students make their own material more accessible, searchable, transformable, and reviewable.

9 Conclusion

This project presents `study-assistant` as a cohesive agent-based system for academic study workflows. The system combines an instruction-driven agent with OCR, RAG, and TTS tools, each of which has a standalone command-line implementation. The resulting architecture supports a practical path from source documents to extracted text, searchable study material, generated revision artefacts, and spoken audio.

The central engineering contribution is the separation between orchestration and processing. `study-assistant`, `ocr-tool`, `rag-tool`, and `tts-tool` define the interaction and workflow contracts. `pdfocr`, `chunkvec`, and `chunktts` implement the core operations in Nim. The custom `relay`, `jsonx`, and `openai` libraries provide the shared transport, JSON, and model-API foundation.

9.1 Key Results

The OCR subsystem demonstrates the value of bounded concurrency. On the recorded 72-page benchmark, `pdfocr` completed all pages successfully in 19.93 seconds at `max_inflight=32`, compared with 316.66 seconds for the sequential baseline. The output contract held: 72/72 page results, strict page order, no retries, and exit status `0`.

The 68-page scientific OCR benchmark positions `allenai/olmOCR-2-7B-1025` as a strong recall-oriented model in the evaluated set. This supports its use in a study-assistant context, where omissions in extracted educational content can reduce the quality of downstream notes, quizzes, and explanations.

9.2 Limitations

The system depends on remote model providers for OCR, embeddings, and speech synthesis. This introduces external variability in latency, availability, pricing, and model

behavior. The empirical OCR results are recorded operational measurements, not universal performance guarantees.

Strict ordering is also a deliberate trade-off. It improves composability and preserves document structure, but it can introduce head-of-line blocking when an early page or chunk is slow. The architecture accepts this cost because ordered educational material is easier to verify, store, retrieve, and transform.

Privacy is another practical limitation. Documents sent to remote inference providers must be suitable for that processing model. Sensitive material requires appropriate provider controls or a local backend.

9.3 Further Work

The main areas for further engineering are deterministic mocked transport tests, broader benchmark corpora, richer retrieval evaluation, optional provider/model selection policies, and stronger workflow auditability. The workflow evaluation specifically motivates run manifests, OCR validation for formulae and diagram-derived text, recorded retrieval evidence, metadata-aware chunking, and validation passes for generated study artefacts. These improvements should preserve the existing architectural boundary: the agent chooses workflows, tool definitions constrain safe use, and core tools enforce stable processing contracts.

References

Chen, X., Jin, L., Zhu, Y., Luo, C., & Wang, T. (2022). Text recognition in the wild: A survey. *ACM Computing Surveys*, *54*(2), 1–35. https://doi.org/10.1145/3440756

curl. (2026, February 28). *libcurl: Multi interface overview*. https://curl.se/libcurl/c/libcurl-multi.html

Dunlosky, J., Rawson, K. A., Marsh, E. J., Nathan, M. J., & Willingham, D. T. (2013). Improving students' learning with effective learning techniques: Promising directions from cognitive and educational psychology. *Psychological Science in the Public Interest*, *14*(1), 4–58. https://doi.org/10.1177/1529100612453266

Geralis, A. (2026, March 1). *OCR_benchmarks_academic*. https://github.com/planetis-m/OCR_benchmarks_academic

Google. (2026b, May 3). *Generate Audio Overview in NotebookLM*. Notebooklm Help. https://support.google.com/notebooklm/answer/16212820

Google. (2026a, May 3). *Learn about NotebookLM*. Notebooklm Help. https://support.google.com/notebooklm/answer/16164461

JSON Lines. (2026, February 28). *JSON Lines*. https://jsonlines.org/

Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W.-t. (2020). Dense passage retrieval for open-domain question answering. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 6769–6781. https://doi.org/10.18653/v1/2020.emnlp-main.550

Kasneci, e. a., E. (2023). ChatGPT for good? On opportunities and challenges of large language models for education. *Learning and Individual Differences*, *103*, Article102274. https://doi.org/10.1016/j.lindif.2023.102274

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems 33*.

Liu, X., Meng, G., & Pan, C. (2019). Scene text detection and recognition with advances in deep learning: A survey. *International Journal on Document Analysis and Recognition*, *22*(2), 143–162. https://doi.org/10.1007/s10032-019-00320-5

Mayer, R. E. (2009). *Multimedia learning* (2nd ed.). Cambridge University Press.

Oord, e. a. van den, A. (2016). WaveNet: A generative model for raw audio. *Arxiv*. https://doi.org/10.48550/arXiv.1609.03499

Poznanski, e. a., J. (2025). olmOCR: Unlocking trillions of tokens in PDFs with vision language models. *Arxiv*. https://doi.org/10.48550/arXiv.2502.18443

Poznanski, J., Soldaini, L., & Lo, K. (2025). olmOCR 2: Unit test rewards for document OCR. *Arxiv*. https://doi.org/10.48550/arXiv.2510.19817

Roediger, I., H. L., & Karpicke, J. D. (2006). Test-enhanced learning: Taking memory tests improves long-term retention. *Psychological Science*, *17*(3), 249–255. https://doi.org/10.1111/j.1467-9280.2006.01693.x

Shen, J., Pang, R., Weiss, R. J., Schuster, M., Jaitly, N., Yang, Z., Chen, Z., Zhang, Y., Wang, Y., Skerry-Ryan, R., Saurous, R. A., Agiomyrgiannakis, Y., & Wu, Y. (2018). Natural TTS synthesis by conditioning WaveNet on mel spectrogram predictions. *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing*, 4779–4783. https://doi.org/10.1109/ICASSP.2018.8461368

Smith, R. (2007). An overview of the Tesseract OCR engine. *Proceedings of the 9th International Conference on Document Analysis and Recognition*, 629–633.

sqliteai. (2026, May 3). *SQLite vector extension: API reference*. https://github.com/sqliteai/sqlite-vector/blob/main/API.md

Standardization. (2020). *Document management: Portable document format: Part 2: PDF 2.0* (Issues ISO32000–2:2020).

Steenbergen-Hu, S., & Cooper, H. (2014). A meta-analysis of the effectiveness of intelligent tutoring systems on college students' academic learning. *Journal of Educational Psychology*, *106*(2), 331–347. https://doi.org/10.1037/a0034752

Subramani, N., Matton, A., Greaves, M., & Lam, A. (2020). A survey of deep learning approaches for OCR and document understanding. *Arxiv*. https://doi.org/10.48550/arXiv.2011.13534

Thea Study. (2026, May 3). *Study with Thea*. https://www.theastudy.com/home

Appendix A

User Manual

This appendix provides practical usage instructions for the study-assistant tool suite. It covers the agent workflow and the standalone command-line tools used for OCR, retrieval, and text-to-speech generation.

Agent Workflow

Use `study-assistant` when the goal is to transform prepared study material into one of the supported outputs:

- verbatim transcription;
- lecture-style explanation;
- ELI5 explanation;
- flashcards;
- Mermaid mind map;
- quiz with answer key;
- essay prompts and sample answers;
- exam-focused study notes.

The agent requires source material that is already text or has been extracted through the OCR workflow.

OCR Workflow

OCR selected PDF pages:

```
pdfocr INPUT.pdf --pages:"1,4-6,12" > results.jsonl
```

OCR all pages:

```
pdfocr INPUT.pdf --all-pages > results.jsonl
```

The `--pages` selector uses 1-based page numbering and supports single pages, inclusive ranges, and comma-separated combinations. Exactly one selector mode must be supplied.

On non-fatal completion, stdout contains JSON Lines only, with one object per selected page in strict page order.

Success example:

```
{"page":12,"status":"ok","attempts":1,"text":"..."}
```

Error example:

```
{
  "page": 12,
  "status": "error",
  "attempts": 3,
  "error_kind": "Timeout",
  "error_message": "...",
  "http_status": 504
}
```

RAG Store Workflow

Prepare a marked text file using `<chunk ...>` markers:

```
<chunk page=4 label="Embeddings">
Embeddings map text into vectors where similar meanings stay close.

<chunk page=5 label="Vector Search">
Nearest-neighbour search compares a query vector against stored vectors.
```

Ingest it:

```
cvstore --doc=notes-course --kind=source --source=course/notes.md
notes.txt
```

The `doc` value should be stable so that retrieval can target the same logical material.

RAG Search Workflow

Query stored material:

```
cvquery --doc=notes-course --kind=source "How do embeddings help search?"
```

Optional filters include `--doc`, `--kind`, `--page`, and `--label`, subject to the tool's validation rules.

TTS Workflow

Prepare text with <bk> boundaries:

```
Introduction paragraph.<bk>
This should become the second spoken section.<bk>
Closing section.
```

Generate audio:

```
chunktts input.txt output.opus
```

The output is one final `.opus` file. Logs and fatal errors are written to stderr.

Configuration

The core tools support optional `config.json` files beside their executables. API keys can be supplied through `DEEPINFRA_API_KEY` or through tool configuration. The environment variable takes precedence when present and non-empty.

Typical configurable values include endpoint URLs, model names, concurrency limits, retry counts, timeouts, OCR render settings, embedding dimensions, and TTS voice/speed.

Exit Codes

The command-line tools use stable exit codes:

- `0`: requested work succeeded;
- `2`: processing completed with permanent item failures where the tool contract permits this result;

- 3: fatal startup or runtime failure.

Appendix B

Installation and Build Guide

This appendix summarises installation and build requirements for the unified tool suite.

Prebuilt Binaries

The core tools are designed to be distributed as platform-specific command-line binaries:

- `pdfocr`;
- `cvstore` and `cvquery`;
- `chunktts`.

Runtime dependencies vary by tool and platform:

- `pdfocr`: `libcurl` and `libwebp`; the release archive also includes the PDFium runtime library, which must remain beside the executable.
- `cvstore` and `cvquery`: `libcurl`, SQLite, and the platform `sqlite-vector` runtime library (`vector.so`, `vector.dylib`, or `vector.dll`) beside the executables.
- `chunktts`: `libcurl` and `libsndfile`.

Typical Linux runtime packages are `libcurl4`, `libwebp7`, `sqlite3`, and `libsndfile1`, depending on which tools are installed. On macOS, install `curl`, `webp`, `sqlite`, and `libsndfile` with Homebrew as needed. Windows release archives bundle the required DLLs; keep those DLLs in the same directory as the `.exe` files.

Source Build Overview

The implementations are Nim projects. Source builds require:

- Nim;

- Atlas or Nimble dependency resolution as documented by each repository;
- platform development packages for the relevant native libraries; and
- downloaded native runtime artefacts where required, specifically PDFium for `pdfocr` and `sqlite-vector` for `cvstore` and `cvquery`.

On Linux, the development packages used by the project workflows are `libcurl4-openssl-dev`, `libwebp-dev`, `sqlite3`, and `libsndfile1-dev`. On macOS, the workflows use Homebrew packages `curl`, `webp`, `sqlite`, and `libsndfile`. On Windows, the workflows use vcpkg packages such as `curl[http2,ssl,c-ares]`, `libwebp`, `sqlite3`, and `libsndfile`.

The common build shape is:

```
atlas install
nim c -d:release -o:TOOL src/ENTRYPOINT.nim
```

Representative outputs are:

```
nim c -d:release -o:pdfocr src/app.nim
nim c -d:release -o:cvstore src/cvstore.nim
nim c -d:release -o:cvquery src/cvquery.nim
nim c -d:release -o:chunktts src/app.nim
```

Shared Libraries

The processing tools depend on the custom libraries:

- `relay` for HTTP;
- `jsonx` for JSON;
- `openai` for OpenAI-compatible API schemas and helpers.

These libraries are resolved as Nim dependencies in the tool repositories.

Testing

Each repository exposes a Nim test task. The common command form is:

```
nim test tests/ci.nims
```

Live end-to-end runs require model-provider credentials. Deterministic unit and integration tests cover local parsing, retry, ordering, JSON, vector, and audio-validation behavior where possible.

Appendix C

Benchmarks and Test Cases

This appendix consolidates benchmark and test artefacts supporting Chapter Section 6.

OCR Throughput Benchmark

The throughput benchmark uses a fixed 72-page slide deck PDF included in the OCR repository test assets.

Recorded results:

- Bounded concurrency (`max_inflight=32`): 19.93 seconds total runtime; 3.61 pages/s; 72/72 pages succeeded; exit status `0`.
- Sequential baseline (`K=1`): 316.66 seconds total runtime; 0.23 pages/s; 72/72 pages succeeded; exit status `0`.

Derived metrics:

Metric	Value
Speedup	15.89x
Absolute time reduction	296.73 s
Relative time reduction	93.71%

Table 3.1: Derived OCR throughput metrics

Recorded OCR Implementation Comparison

A side-by-side recorded comparison uses the same 72-page test PDF and `--all-pages` selection mode.

Metric	External reported run	This project recorded run
Wall time	17.04 s	28.23 s
Throughput	4.23 pages/s	2.55 pages/s
Peak RSS	71.51 MiB	86.29 MiB

Metric	External reported run	This project recorded run
Exit status	0	0
Output validation	full ordered JSONL	72/72 ok, 0 retries, strict page order

Table 3.2: Recorded OCR implementation comparison

These observations are run-specific because OCR requests depend on live network and provider conditions. Peak RSS is the appropriate memory metric for process-level comparison; allocator-state snapshots are not equivalent to process peak memory.

Scientific OCR Benchmark

The model-comparison benchmark uses a fixed 68-page academic dataset with locked human gold labels. The benchmark input is a consolidated PDF assembled from 34 source PDFs, two pages per source. The selected pages include equations, multi-column layouts, diagrams, and tables. (Geralis, 2026)

All measured models completed 68/68 pages.

Model	CER	WER	ReadingOrderF1	MathF1
`PaddlePaddle/PaddleOCR-VL-0.9B`	0.4634	0.4732	0.3751	0.7248
`allenai/olmOCR-2-7B-1025`	0.4682	0.4893	0.3252	0.6743
`deepseek-ai/DeepSeek-OCR`	0.5862	0.6512	0.1452	0.4684

Table 3.3: Scientific OCR benchmark accuracy results

Model	Total cost (USD)	Cost/page (USD)
`PaddlePaddle/PaddleOCR-VL-0.9B`	0.0420	0.0006180
`allenai/olmOCR-2-7B-1025`	0.0214	0.0003142
`deepseek-ai/DeepSeek-OCR`	0.0041	0.0000597

Table 3.4: Scientific OCR benchmark cost results

Recorded conclusions:

- strict-accuracy winner: `PaddlePaddle/PaddleOCR-VL-0.9B`;
- recall-oriented winner: `allenai/olmOCR-2-7B-1025`;
- cost-first and balanced-score winner: `deepseek-ai/DeepSeek-OCR`.

Repository Test Groups

The deterministic test groups are:

- `pdfocr`: page selection, request-id codec, retry queue, retry/error mapping, JSON result schema;
- `chunkvec`: chunk parsing, runtime configuration, request-id codec, embeddings client, SQLite/vector integration;
- `chunktts`: chunk splitting, request-id codec, retry/error mapping, speech client, audio wrapper, pipeline integration;
- `relay`: lifecycle contracts, ordering contract, headers, request-body round trip, batch helpers;
- `jsonx`: parsing, compliance, number handling, object mapping;
- `openai`: chat, embeddings, audio speech, retry helpers.

Appendix D

Focused Code Listings

The implementation relies on a small number of mechanisms that determine the behaviour of the OCR, RAG, TTS, transport, and model-client layers. The following excerpts document deterministic request identity, ordered OCR output, retry-aware completion handling, strict RAG chunk parsing, filtered vector search, final audio publication, transport worker control, and OpenAI-compatible request construction.

Request Identifier Packing (`pdfocr/src/pdfocr/request_id_codec.nim`)

Request identifiers provide the main link between asynchronous network completions and logical work items. The same design appears in the OCR, RAG, and TTS pipelines.

```
const
  RequestAttemptBits* = 16
  RequestAttemptMask = (1'u64 shl RequestAttemptBits) - 1'u64
  RequestAttemptMax* = int(RequestAttemptMask)
  RequestSeqIdBits = 63 - RequestAttemptBits
  RequestSeqIdMax* = (1'i64 shl RequestSeqIdBits) - 1'i64

proc ensureRequestIdCapacity*(selectedCount: int; maxAttempts: int) =
  if selectedCount > 0 and selectedCount.int64 - 1 > RequestSeqIdMax:
    raise newException(ValueError,
      "selected page count exceeds request-id packing capacity")
  if maxAttempts > RequestAttemptMax:
    raise newException(ValueError,
      "max attempts exceeds request-id packing capacity")

proc packRequestId*(seqId: int; attempt: int): int64 =
  if seqId < 0 or seqId.int64 > RequestSeqIdMax:
```

```
    raise newException(ValueError, "seqId out of range for request id")
  if attempt < 1 or attempt > RequestAttemptMax:
    raise newException(ValueError, "attempt out of range for request id")
  let packed = (uint64(seqId) shl RequestAttemptBits) or uint64(attempt)
  result = int64(packed)

proc unpackRequestId*(requestId: int64): tuple[seqId, attempt: int] =
  let packed = cast[uint64](requestId)
  result = (
    seqId: int(packed shr RequestAttemptBits),
    attempt: int(packed and RequestAttemptMask)
  )
```

Ordered OCR Emission (`pdfocr/src/pdfocr/pipeline.nim`)

The OCR pipeline enforces strict page order through a staged result array. Results may complete out of order, but emission is permitted only for the next staged sequence id whose status is no longer pending.

```
proc emitPageResult(output: Stream; value: PageResult): bool =
  output.writeJson(value)
  streams.write(output, '\n')
  result = value.status == PageOk

proc flushOrderedResults(state: var PipelineState) =
  while state.nextEmitSeqId < state.staged.len and
      state.staged[state.nextEmitSeqId].status != PagePending:
    if not emitPageResult(state.output,
state.staged[state.nextEmitSeqId]):
      state.allSucceeded = false
    state.staged[state.nextEmitSeqId] = default(PageResult)
    inc state.nextEmitSeqId
    dec state.remaining
```

OCR Completion Classification (`pdfocr/src/pdfocr/pipeline.nim`)

The OCR reliability model classifies each asynchronous completion as retryable work, a successful page result, or a structured page error. The classification combines transport status, HTTP status, response parsing, retry limits, and request identity.

```
proc processResult(cfg: RuntimeConfig; item: RequestResult; maxAttempts:
int;
    retryPolicy: RetryPolicy; state: var PipelineState) =
  let requestId = item.response.request.requestId
  let meta = unpackRequestId(requestId)
  let seqId = meta.seqId
  let attempt = meta.attempt
  dec state.inFlightCount

  if shouldRetry(item, attempt, maxAttempts):
    let delayMs = retryDelayMs(state.rng, attempt, retryPolicy)
    state.retryQueue.addRetry(RetryItem(
      seqId: seqId,
      attempt: attempt + 1,
      dueAt: getMonoTime() + initDuration(milliseconds = delayMs)
    ))
  else:
    let pageNumber = cfg.selectedPages[seqId]
    if item.error.kind != teNone or not
isHttpSuccess(item.response.code):
      let finalError = classifyFinalError(item)
      state.staged[seqId] = errorPageResult(
        page = pageNumber,
        attempts = attempt,
        kind = finalError.kind,
        message = finalError.message,
        httpStatus = finalError.httpStatus
      )
```

```
    else:
      var text = ""
      if parseOcrText(item.response.body, text):
        state.staged[seqId] = okPageResult(
          page = pageNumber,
          attempts = attempt,
          text = text
        )
      else:
        state.staged[seqId] = errorPageResult(
          page = pageNumber,
          attempts = attempt,
          kind = ParseError,
          message = "failed to parse OCR response"
        )
    state.cachedPayloads[seqId] = default(CachedPayload)
    dec state.activeCount
```

Strict RAG Chunk Parsing (`chunkvec/src/chunkvec/input_chunks.nim`)

The quality of retrieval depends on controlled chunk boundaries and metadata. The parser rejects missing markers and empty chunks instead of silently ingesting ambiguous content.

```
proc parseInputChunks*(text: string): seq[InputChunk] =
  var pos = skipWhitespace(text)

  while pos < text.len:
    if not markerAtLineStart(text, pos, ChunkMarkerPrefix):
      failParse("missing <chunk ...> marker")

    var chunk: InputChunk
    let markerLen = parseChunkMarker(text, chunk, pos)
    if markerLen == 0:
```

```
      failParse("expected <chunk ...> marker")

    let bodyStart = pos + markerLen
    let nextMarkerPos = findNextMarker(text, bodyStart,
ChunkMarkerPrefix)
    let bodyBounds = trimChunkBounds(text, bodyStart, nextMarkerPos)
    if bodyBounds.a >= bodyBounds.b:
      failParse("chunk body is empty")

    chunk.text = text[bodyBounds]
    result.add(chunk)

    pos = nextMarkerPos
```

Filtered Vector Search (`chunkvec/src/chunkvec/chunk_store.nim`)

Semantic vector search and metadata filtering operate in the same query. Vector distance supplies semantic ranking, while document, kind, page, and label filters constrain the retrieval scope used by the RAG workflow.

```
proc runFilteredSearch(db: DbConn; queryVector: seq[float32]; filters:
SearchFilters;
    topK: int): seq[SearchResult] =
  var query =
    """SELECT
  c.id,
  v.distance,
  c.source,
  c.text,
  c.doc_id,
  c.kind,
  c.page,
  c.label
FROM """ & TableName & """ AS c
```

```
JOIN vector_quantize_scan('""" & TableName & "', '" &
    EmbeddingColumn & """', ?) AS v
  ON c.id = v.rowid
""" # """

  var haveWhereClause = false
  if filters.docId.len > 0:
    query.add("WHERE c.doc_id = ?\n")
    haveWhereClause = true
  if filters.kind != none:
    addWherePrefix()
    query.add("c.kind = ?\n")
  if filters.page != NoPageFilter:
    addWherePrefix()
    query.add("c.page = ?\n")
  if filters.labelSubstring.len > 0:
    addWherePrefix()
    query.add("instr(" & normalizedLabelExpr("c.label") & ", ?) > 0\n")

  query.add("ORDER BY v.distance ASC, c.id ASC\n")
  query.add("LIMIT ?;")
```

Final TTS Artefact Publication (`chunktts/src/chunktts/pipeline.nim`, `sndfile_wrap.nim`)

The TTS pipeline publishes a final `.opus` artefact only when every chunk succeeds. Audio finalisation also validates that all decoded chunks share compatible sample rates and channel counts.

```
proc runPipeline*(cfg: RuntimeConfig; chunks: seq[string]; client:
Relay): bool =
  let total = chunks.len
  let maxInFlight = max(1, cfg.networkConfig.maxInflight)
```

```
  let maxAttempts = max(1, cfg.networkConfig.maxRetries + 1)
  let retryPolicy = defaultRetryPolicy(maxAttempts = maxAttempts)
  ensureRequestIdCapacity(total, maxAttempts)

  var state = initPipelineState(total)

  while state.remaining > 0:
    submitDueRetries(cfg, chunks, maxInFlight, state)
    submitFreshAttempts(cfg, chunks, maxInFlight, state)
    startBatchIfAny(client, state)
    let drained = drainReadyResults(client, maxAttempts, retryPolicy,
state)

    if state.remaining > 0 and not drained:
      waitForProgress(client, maxInFlight, maxAttempts, retryPolicy,
state)

  if state.allSucceeded:
    writeOpusFile(cfg.outputPath, state.decodedChunks)

  result = state.allSucceeded
```

```
proc writeOpusFile*(path: string; chunks: openArray[DecodedAudio]) =
  if chunks.len == 0:
    raise newException(ValueError, "cannot write zero audio chunks")

  let sampleRate = chunks[0].sampleRate
  let channels = chunks[0].channels
  let audioFile = openAudioFileForWrite(
    path,
    sampleRate,
    channels,
    SF_FORMAT_OGG or SF_FORMAT_OPUS
  )
```

```
  for chunk in chunks:
    if chunk.sampleRate != sampleRate:
      raise newException(ValueError, "chunk sample rates do not match")
    if chunk.channels != channels:
      raise newException(ValueError, "chunk channel counts do not match")
    writeDecodedAudio(audioFile, chunk)
```

Relay Worker Control Loop (`relay/src/relay.nim`)

The `relay` client uses a worker thread to dispatch queued requests, drive in-flight transfers, process completed responses, and honour abort requests. The same transport control loop is shared by the OCR, RAG, and TTS tools.

```
proc workerMain(clientPtr: ptr RelayObj) {.thread, raises: [].} =
  let client = cast[Relay](clientPtr)
  while true:
    dispatchQueuedRequests(client)

    acquire(client.lock)
    let hasInflight = client.inFlight.len > 0
    let shouldAbort = client.abortRequested
    release(client.lock)

    if shouldAbort:
      acquire(client.lock)
      flushCanceledLocked(client, "Canceled in abort")
      release(client.lock)
      break

    if hasInflight:
      if not runEasyLoop(client):
        break
      processDoneMessages(client)
```

```
    elif not waitForWorkOrClose(client):
      break

  acquire(client.lock)
  client.workerRunning = false
  signal(client.resultCond)
  release(client.lock)
```

OpenAI-Compatible Request Boundary (`openai/src/openai/chat.nim`, `openai/src/openai/http.nim`)

The OpenAI-compatible client layer separates model schema construction from HTTP execution. The `openai` library builds `RequestSpec` values, and `relay` performs the transport work.

```
proc chatRequest*(cfg: OpenAIConfig; params: ChatCreateParams;
    requestId = 0'i64; timeoutMs = 0;
    headers: sink HttpHeaders = emptyHttpHeaders()): RequestSpec =
  jsonPostRequest(cfg, params, requestId, timeoutMs, headers)

proc chatAdd*(batch: var RequestBatch; cfg: OpenAIConfig;
    params: ChatCreateParams; requestId = 0'i64; timeoutMs = 0;
    headers: sink HttpHeaders = emptyHttpHeaders()) =
  jsonPostAdd(batch, cfg, params, requestId, timeoutMs, headers)

proc jsonPostRequest*[T](cfg: OpenAIConfig; params: T;
    requestId = 0'i64; timeoutMs = 0;
    headers: sink HttpHeaders = emptyHttpHeaders()): RequestSpec =
  RequestSpec(
    verb: hvPost,
    url: cfg.url,
    headers: cfg.withDefaultHeaders(headers),
    body: toJson(params),
```

```
    requestId: requestId,
    timeoutMs: timeoutMs
  )
```

Administrative Data

University of Nicosia, Department of Computer Science, School of Sciences and Engineering.

- Prepared by: Antonis Geralis, May 2026.
- Project advisor: Ioannis Katakis.
- Examiners: Athena Stassopoulou; Vaso Stylianou.
- Project title: An Agent-Based Study Assistant with OCR, Retrieval, and Speech.